HANDBUCH

Testhandbuch

Wie ein Test aufgebaut ist, wie er abläuft und wie man sein Ergebnis liest

Dieses Handbuch beschreibt das Testkonzept des Projekts: Trommeldatei, Tastenfolge und Prüfungen, die Testmarken, die formale Prüfung vor dem Start, das Protokoll und die Ein- und Ausgabe. Alle gezeigten Läufe sind echte Läufe des Python-Interpreters. Es ist redaktionell aus der Quelle gefasst und ergänzt sie um Beispiele; verbindlich bleibt die Quelle.

Quelle	`C8205-Interpreter-Durchführung_eines_Tests.md`	0.95-001
Stand	21.10.2026	
Umfang	40 Seiten, A4 hoch	
Erzeugt mit	`build_testhandbuch.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Lesehinweise

Dieses Dokument beschreibt das Testkonzept des Projekts: Jeder simulierte „Rechnerlauf“ ist ein Test, unabhängig davon, ob ihn der Python-Interpreter oder eine KI (Claude-Interpreter) ausführt. Ein Test besteht aus einer Trommeldatei (Speicherinhalt, Testmarken), einer Tastenfolge (Bedienhandlungen) und optional Prüfungen (Sollwerte).

Quelle und Stand. `C8205-Interpreter-Durchführung_eines_Tests.md` (Version 0.95-001), ergänzt um die Beobachtungen aus Läufen mit `c8205_interp.py` (Version 0.95-001) und mit dem Assembler `c82ass` (Version 0.2). Alle gezeigten Bildschirm- und Protokollauszüge des Interpreters sind echte Läufe vom 21.10.2026 auf leerer Trommel; nur der Auszug aus dem Assemblerlauf (3.3) stammt vom 05.10.2026. Kürzungen sind mit „...“ gekennzeichnet, lange Pfade sind verkürzt.

> **Hinweis** Abgrenzung: Wie man den Interpreter aufruft (Schalter, Fehlermeldungen), steht im PDF „Bedienungsanleitung Python-Interpreter“. Dieses Handbuch erklärt, was ein Test ist und warum er so aufgebaut ist; für Rechner und Bedienpult siehe „Rechnerbeschreibung“ und „Bedienpult und Peripherie“. Die Beispieltests dieses Handbuchs (`bspA`, `bspB`, `pruef-wahr`, `vorlage` u. a.) liegen in der Zip-Datei `Handbuch-Tests_Testhandbuch.zip` (Ordner `Handbuch-Tests/`) und gehören nicht zu den Regressionstests in `Tests/` (11.1).

> **Achtung** Grundsatz „laut scheitern“: Bei einer Unklarheit, einem Widerspruch oder einer fehlenden Festlegung wird der Test angehalten und das gemeldet – es wird nichts stillschweigend angenommen oder repariert.

Legende dieses Dokuments

Zeichen	Bedeutung
`Trommeldatei:` `Tastenfolge:` `Prüfungen:`	die drei Teile einer Testdatei (Kapitel 2)
`#DOK` `#SPEI` `#SPUR` `#PROT` `#AC=` `#BZ=` `#INTERRUPT` `#STARTE`	Testmarke: Zeile mit #, die nur für den Test gilt und in der Hardware keine Entsprechung hat (3.2)
`--leser1` `--leser2` `--stanzer1` `--sw-ausgabe` `--k0`	Geräte-Zeile: ordnet der Testdatei ein virtuelles Gerät zu (3.4)
Python- / Claude-Interpreter	die beiden Ausführenden desselben Testformats (1.1, Kapitel 10)
Hinweis / **Achtung**	blauer bzw. orangefarbener Balken: nützliche Ergänzung bzw. Stelle, an der Tests scheitern oder Regelwerk und Interpreter abweichen
grauer Block	Dateiinhalt oder Terminalausgabe in Festbreitenschrift; Eingaben stehen hinter $
Abbildungen	**grün** = positiv / weiter, **rot gestrichelt** = Fehler / Test startet nicht, **grau punktiert** = Verweis, schwarz = neutraler Ablauf

Zuerst steht die Kurzanleitung „Test starten“ (Claude-Interpreter ohne Python und Python-Interpreter). Die Kapitel folgen dem Lebenslauf eines Tests: Einordnung (1), Aufbau (2), die drei Teile Trommeldatei (3, mit den Regeln der persistenten Trommel in 3.5 und den Anfangswerten in 3.6), Tastenfolge (4, mit `#STARTE` und `#INTERRUPT`) und Prüfungen (5), dann die formale Prüfung vor dem Start (6), die Diagnose (7), das Protokoll (8), Ein- und Ausgabe (9), der Claude-Interpreter (10; dort in 10.3 auch die Aufstellung der Dateien, die der Python-Interpreter braucht), die Referenztests (11, mit den Tests der BIS-Bausteine) sowie Checkliste und Vorlage (12).

Inhaltsverzeichnis

Kurzanleitung: Test starten . . . 5
A Mit dem Claude-Interpreter (ohne Python) . . . 5
B Mit dem Python-Interpreter . . . 5

1 Einordnung und Voraussetzungen . . . 6
1.1 Zwei Ausführende, ein Format . . . 6
1.2 Ein Test im Überblick . . . 6

2 Aufbau einer Testdatei . . . 7

3 Schritt 1: die Trommeldatei . . . 7
3.1 Zeilenformat . . . 7
3.2 Testmarken . . . 8
3.3 Testauftrag aus einem Assemblerlauf . . . 8
3.4 Geräte-Zeilen --leser1, --leser2, --stanzer1, --sw-ausgabe und --k0 . . . 9
3.5 Die Trommel als persistente Datei (Python-Interpreter) . . . 10
3.6 Anfangswerte der Register: #AC= und #BZ= . . . 12

4 Schritt 2: die Tastenfolge . . . 13
4.1 Beispiel A: Ansprung des Speicherplatzes 0000 (direkt) . . . 13
4.2 Beispiel B: Ansprung eines beliebigen Speicherplatzes (hier 0004) . . . 14
4.3 Festlegungen zu den Tasten M, Ü, E, ST, BS, SP . . . 14
4.4 Wo der Lauf nach „R“ beginnt . . . 15
4.5 Weitere typische Bedienfolgen (Fälle 1 bis 6) . . . 16
4.6 Das Makro #STARTE . . . 16
4.7 Unterbrechung des Programmlaufs: #INTERRUPT . . . 18

5 Schritt 3 (optional): Prüfungen . . . 19
5.1 Aufbau . . . 19
5.2 Auswertung und Ausgabe . . . 20
5.3 Typische Sollwerte . . . 21
5.4 Dateiprüfungen für Stanzer und Schreibwerk . . . 21
5.5 Erwartetes Ergebnis „falsch“: Prüfungen: False . . . 22

6 Formale Gültigkeitsprüfung vor dem Start . . . 23

7 Diagnose beim Leerbefehl 164 . . . 25
7.1 Auslösebedingung . . . 26
7.2 Ablauf . . . 26

8 Protokoll und Protokolltiefe . . . 27
8.1 Inhalt der Protokolldatei (Stufe 2) . . . 27
8.2 Eine Knotenzeile lesen . . . 28

9 Ein- und Ausgabe in Tests . . . 28
9.1 Lochstreifendateien . . . 29
9.2 Schreibwerk . . . 29
9.3 Gegenüberstellung der Ausführenden . . . 30

10 Den Claude-Interpreter einsetzen . . . 30
10.1 Was die KI braucht . . . 30
10.2 Vorgehen . . . 30
10.3 Was der Python-Interpreter braucht . . . 31

11 Referenztests und Regression . . . 33
11.1 Testbestand und Gesamtlauf . . . 33
11.2 Referenztests zu den Tastenfolgen (tk-001 bis tk-008) . . . 35
11.3 Vollständigkeitstest . . . 35
11.4 Tests der BIS-Bausteine: das Minimalprogramm läuft zuerst . . . 35

12 Checkliste und Vorlage . . . 38

12.1 Checkliste für neue Tests 38
12.2 Vorlage 38

Anhang A: Begriffe 39

Anhang B: Verweise 40

Kurzanleitung: Test starten

Ein Test besteht aus der Testdatei (Kapitel 2 bis 5) und, wenn er Geräte nutzt, aus Dateien für diese Geräte. Ausgeführt wird er auf einem von zwei Wegen: durch eine KI ohne Python (Weg A) oder durch `c8205_interp.py` (Weg B). Beide arbeiten denselben Knotengraphen mit demselben Testformat ab und führen das Protokoll im selben Format.

A Mit dem Claude-Interpreter (ohne Python)

1. **Unterhaltung beginnen**, in der Sie Dateien hochladen können. Die KI führt den Knotengraphen von Hand aus; es wird kein Programm gestartet.
2. **Regelwerk hochladen**: die Dateien aus Kapitel 10.1 (Ordner `markdown-dateien/`; die Faustregel dort sagt, was Sie weglassen können).
3. **Testdatei bereitstellen**: nach Kapitel 3 bis 5 schreiben (Vorlage in 12.2) und hochladen oder in die Nachricht kopieren. Geräte-Zeilen wie `--leser1` gelten hier als virtuelle Zuordnung (3.4). Lochstreifendateien legen Sie bereit; die KI fordert sie an, wenn ein Eingabebefehl sie braucht (9.1). Eine Trommeldatei braucht die KI nicht: Zellen, die der Test nicht nennt, gelten als 0 (3.5).
4. **Startanweisung** in den Chat schreiben, zum Beispiel wie im Block darunter. Der Text ist ein Vorschlag und nicht Teil des Regelwerks.
5. **Ergebnis lesen**: Endzeile, Ergebnisprüfung `Test wahr` oder `Test falsch` und das Protokoll im Format der Stufe 2 (Kapitel 8). Rückkehrcodes (5.2) gibt es nur beim Python-Aufruf. Stanzer- und Schreibwerk-Ausgabe stellt die KI am Testende als Datei bereit (9.3). Weicht das Ergebnis vom Python-Lauf ab, siehe 1.1 (Rolle jetzt).

```
Führe den hochgeladenen Test <Testname> als Claude-Interpreter aus, ohne Python oder ein anderes Programm.
Regelwerk sind die hochgeladenen .md-Dateien. Prüfe zuerst die Testdatei formal.
Arbeite dann den Knotengraphen (Eingänge_der_Knoten, Variablentabelle T2 in Beschreibung_der_Knoten)
Knoten für Knoten ab und führe das Protokoll in der Stufe, die #PROT STUFE vorgibt.
Bei jeder Unklarheit, jedem Widerspruch und jeder fehlenden Festlegung: anhalten und rückfragen,
nichts annehmen. Gib am Ende die Endzeile und die Ergebnisprüfung (Test wahr / Test falsch) aus.
```

Vorschlag für die Startanweisung (Weg A); <Testname> durch den Namen der hochgeladenen Testdatei ersetzen.

B Mit dem Python-Interpreter

1. **Voraussetzung**: Python 3, sonst nichts (nur Standardbibliothek). Im Projektordner liegen `c8205_interp.py` und daneben `markdown-dateien/`; der Interpreter liest daraus `Eingänge_der_Knoten.md` und `Beschreibung_der_Knoten.md` (anderer Ort: Schalter `--mdverz`). Außerdem muss die Trommeldatei `Trommel/trommel.txt` vorhanden sein; leer erzeugen mit `python3 erzeuge_leere_trommel.py` (anderer Ort: Schalter `--trommel`; 3.5). Die vollständige Aufstellung aller Dateien und Pfade steht in 10.3.
2. **Testdatei ablegen**: in `Tests/` (die Endung ist frei, der Assembler erzeugt `.test`). Geräte ordnen Sie per Schalter oder Zeile zu (3.4); Dateien dafür liegen üblicherweise unter `Test-virtuelleGeraete/`.
3. **Starten**: direkt oder mit dem Wrapper, der das Protokoll nach `Test-Protokolle/` schreibt. Die Trommel behält ihren Inhalt von Test zu Test und von Aufruf zu Aufruf; für einen Lauf auf leerer Trommel vorher `python3 erzeuge_leere_trommel.py` aufrufen (3.5).
4. **Ergebnis lesen**: Endzeile `Ende: …`, Ergebnisprüfung `Test wahr` oder `Test falsch` und der Rückkehrcode (5.2: 0 wahr, 4 ohne Prüfung, 6 falsch, 8 Laufzeitfehler, 16 Vorab-Prüfung negativ oder Trommeldatei fehlerhaft, 2 ungültiger Aufruf). Knotenzeilen im Protokoll erscheinen mit `--knotentest` und `#PROT STUFE=2` (Kapitel 8).

```
$ python3 erzeuge_leere_trommel.py                  # leere Trommel (alle 4096 Zellen 0)
$ python3 c8205_interp.py --MODUS=TEST Tests/0015-030.txt
$ ./c8205_interpreter.sh 0015-030.txt        # Linux/macOS, Protokoll: Test-Protokolle/0015-030.log
C:\> c8205_interpreter.bat 0015-030.txt      # Windows
$ ./compute_all_tests.sh                     # Sammellauf über Tests/*.txt (11.1)
```

Aufrufe nach README und Bedienungsanleitung. Für dieses Handbuch ausgeführt wurde der Aufruf mit python3; die Wrapper nicht.

Hinweis Alle Schalter, die Geräte und die Fehlermeldungen des Python-Interpreters beschreibt das PDF „Bedienungsanleitung Python-Interpreter“ (Quelle: `Python_Interpreter.md`). Wo sich die beiden Wege bei Ein- und Ausgabe unterscheiden, zeigt die Gegenüberstellung in 9.3. Die Kurzanleitung steht auch als Abschnitt 1b in `Durchführung_eines_Tests.md`.

1 Einordnung und Voraussetzungen

Das Gesamtprojekt beschreibt zwei Aspekte: erstens die Hardware und die in der Elektronik angelegten Beziehungen (Knotenautomat), zweitens davon abweichend oder ergänzend die Annahmen für die interpretative Abarbeitung eines Befehls. Der zweite Aspekt ist die Voraussetzung jedes Tests.

- **Alle Zeiten sind 0.** Zeitliche Abhängigkeiten werden in ihrer Reihenfolge betrachtet, führen aber nicht zu Wartezeiten.
- **Der Rechner ist eingeschaltet.** Jeder Test setzt das voraus.
- **Der Vor-Zustand** vom Einschalten bis zum Beginn der Interpretation der Befehlsschleife am Konnektor B (Knoten 1 des Graphen) ist vollständig bestimmt durch die betätigten Tasten und Schalter und den Trommelinhalt, d. h. den Inhalt der persistenten Trommel, nachdem die Zeilen des Tests auf sie geschrieben wurden (3.5). Register, die der Test nicht nennt, starten mit 0.

1.1 Zwei Ausführende, ein Format

Ein und derselbe Test kann auf zwei Arten ausgeführt werden. Beide benutzen denselben Knotengraphen, dasselbe Testformat und dasselbe Protokollformat.

	Python-Interpreter	Claude-Interpreter (KI)
Wie	`c8205_interp.py` läuft den Knotengraphen deterministisch und automatisiert ab	Eine KI erhält das komplette Regelwerk als Kontext und die Testdatei und arbeitet den Graphen manuell, Knoten für Knoten, ab
Stärke	schnell, reproduzierbar (bei definiertem Trommelstand, 3.5)	erlaubt Rückfragen und Erläuterungen während der Abarbeitung
Schwäche	keine Rückfrage bei unklaren Stellen: der Lauf meldet einen Stub und bricht ab	langsamer, nicht deterministisch reproduzierbar (abhängig von Modell und Sitzung)
Trommel	persistente Datei `trommel.txt`; behält ihren Inhalt über Tests und Aufrufe hinweg	keine Trommeldatei; nicht genannte Zellen gelten als 0 (wie ein Python-Lauf auf leerer Trommel)
Rolle jetzt	Normalbetrieb für laufende Testläufe	unabhängige Gegenprobe: weichen die Ergebnisse ab, liegt ein Fehler in einem von beiden vor – oder eine unklare Stelle im Regelwerk
Rolle bei Erweiterungen	übernimmt neue oder geänderte Befehle erst danach	neue oder geänderte Befehle/Anweisungen werden zuerst hier durchgespielt

Hinweis Diese Aufgabenteilung gilt modellneutral für jede interpretierende KI, nicht nur für Claude. Bei unklaren Regelwerk-Stellen darf die ausführende KI Rückfragen stellen, statt eigenmächtig zu entscheiden. Die Zahl der Rückfragen hängt vom Modell ab.

1.2 Ein Test im Überblick

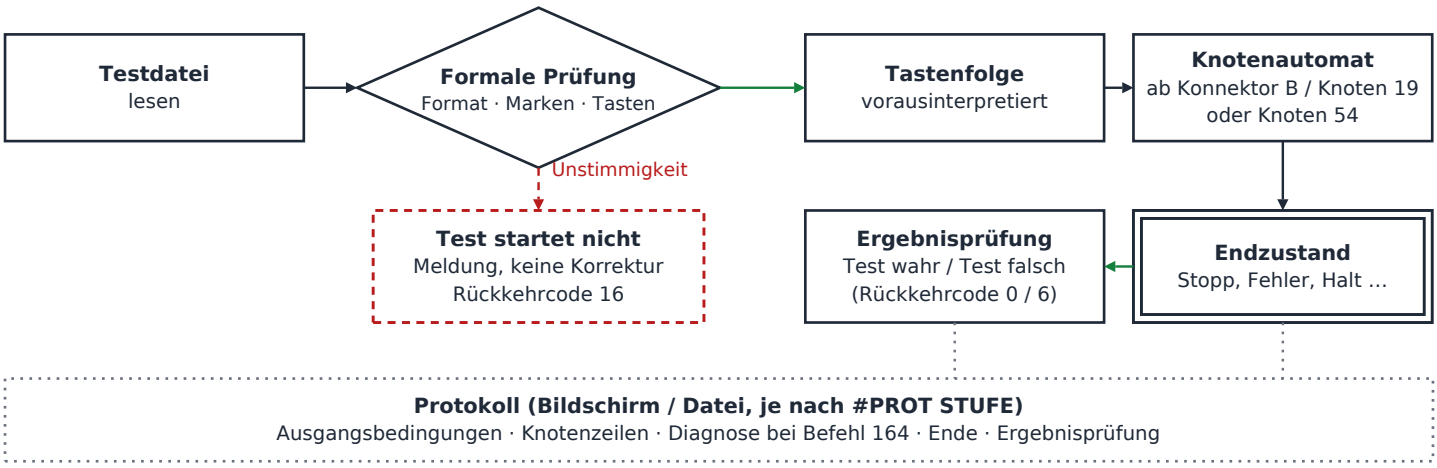


Abbildung 1: Lebenslauf eines Tests. Rot gestrichelt: der Fall, in dem der Test nicht startet („laut scheitern“). Der Startpunkt im Knotenautomaten hängt vom Zustand nach der Tastenfolge ab (4.4).

Der Lauf endet immer mit einer Endzeile; die Ergebnisprüfung erfolgt nur nach ordnungsgemäßem Ende. Zwischendurch, aber nur bei Bedarf: die Diagnose bei Befehl 164 (Kapitel 7) und die Ein-/Ausgabe über Geräte (Kapitel 9).

2 Aufbau einer Testdatei



Abbildung 2: Die drei Teile einer Testdatei (eine Textdatei, die drei Teile stehen in dieser Reihenfolge, Teil 3 darf fehlen). Die Zeilen `Trommeldatei:`, `Tastenfolge:` und `Prüfungen:` leiten jeweils einen Teil ein.

Teil	Pflicht?	Legt fest	Kapitel
Trommeldatei	ja	die Zellen, die vor dem Start auf die Trommel geschrieben werden (ein Ausschnitt; sie überlagern deren Inhalt, 3.5), die Testmarken (`#DOK`, `#SPEI`, `#SPUR`, `#PROT`, `#AC=`, `#BZ=`, `#INTERRUPT`, `#STARTE`) und die Geräte-Zeilen	3
Tastenfolge	ja	Bedienhandlungen am Pult → Ausgangszustand von Registern und Flags vor Konnektor B; mit `#STARTE` entfällt der Abschnitt	4
Prüfungen	nein	Sollwerte für Register, Flags, Speicherzellen und Ausgabedateien nach Testende	5

Hinweis Dateiname: `NNNN-<Operationscodes>.txt` für Testdateien in `Tests/` (z. B. `0004-210_544_020.txt`), `asm_NNNN_<Programmname>.txt` für Testaufträge aus Assemblerläufen. Das Protokoll trägt denselben Namen mit der Endung `.log` in `Test-Protokolle/`.

3 Schritt 1: die Trommeldatei

Der Trommelinhalt, den ein Test braucht, wird vor Beginn jedes Tests als Textdatei mit festem Aufbau übermittelt (Abschnitt `Trommeldatei:` der Testdatei). Er ist nur ein Ausschnitt (adressweise) der Gesamttrommel: Der Interpreter schreibt ihn vor dem Start auf die Trommel und überlagert damit die vorhandenen Werte; Zellen, die er nicht nennt, behalten ihren bisherigen Inhalt (3.5).

3.1 Zeilenformat

```
Adresse: Wert [# Bemerkung]
```

- **Adresse:** 4 Oktalstellen (12 Bit). **Wert:** 11 Oktalstellen (33 Bit), mit Vornullen ergänzt. Alle Zahlen oktal.
- Leerzeilen werden übergangen. Zeilen mit `#` sind Kommentar; ein Kommentar darf – durch mindestens ein Leerzeichen getrennt – hinter einer Adresse:Wert-Zeile stehen und gilt ab dem `#` bis Zeilenende.
- Der Wert besteht aus vier führenden Stellen „beliebig“ (z_0–z_{11}) und sieben Stellen Befehl: `0000 3000 211` = Adresse 3000, Operation 211.

```
# Beginn Programmbefehle
0000: 00003000211
0001: 00003001210
0002: 00000000020
# Beginn Datenbereich
3000: 00000000100 # Startwert der Schleife
3001: 00000000045
```

Achtung Eine nicht genannte Zelle behält ihren bisherigen Inhalt; auf leerer Trommel (und beim Claude-Interpreter) ist das 0. Als Befehl gelesen ist 0 der unbedingte Sprung nach Adresse 0000. Programme müssen deshalb mit einem Stopp-Befehl `00000000020` enden, sonst läuft der Test in eine Endlosschleife und bricht nach dem Knotenlimit ab.

3.2 Testmarken

Testmarken sehen aus wie Kommentare, sind aber keine folgenlosen Kommentare. Sie sind eine von der Hardware abweichende Ergänzung, die ausschließlich der besseren Durchführung der Tests dient.

Marke	Bedeutung	Hinweise
`#DOK-xxxx <Text>`	freier Text zum Test; xxxx oktal. Löst beim Leerbefehl 164 mit der Adresse xxxx die Diagnoseausgabe aus (Kapitel 7)	darf mehrfach mit derselben Nummer vorkommen (Ausgabe in Dateireihenfolge)
`#SPEI-xxxx ssss-eeee`	gibt zusätzlich den Speicherbereich ssss bis eeee aus	ssss und eeee oktal, gültige Spanne (ssss ≤ eeee); mehrfach zulässig; nach Regelwerk nur zu einem vorhandenen `#DOK-xxxx` (Abweichung im Interpreter: Kapitel 6)
`#SPUR-xxxx aaaa`	gibt zusätzlich die ganze Spur aus, in der aaaa liegt (alle 32 Zellen, in physischer Zugriffsreihenfolge der Trommel)	welcher Sektor von aaaa gemeint ist, spielt keine Rolle (Bahn-Nr. = aaaa ohne die unteren 5 Bit); mehrfach zulässig
`#PROT STUFE=n`	Protokolltiefe: 0 / 1 / 2 (Kapitel 8)	höchstens einmal; fehlt sie, gilt Stufe 1
`#AC=` / `#BZ=`	Anfangswert von Akkumulator (11 Oktalziffern) bzw. Befehlszähler (4 Oktalziffern), gesetzt vor der ersten Taste (3.6)	jede höchstens einmal; fehlt sie, startet das Register mit 0
`#STARTE n`	Kurzschreibweise für den Ansprung des Speicherplatzes n; ersetzt die Tastenfolge (4.6)	n oktal, 1 bis 4 Ziffern; höchstens einmal
`#INTERRUPT H …`	unterbricht den Programmlauf nach dem Befehl der Zeile davor und bedient eine weitere Tastenfolge (4.7)	nur zwischen zwei Befehlszeilen; erste Taste H; je Befehlszeile höchstens einmal

Reihenfolge der Ausgabe zu einem xxxx: zuerst alle `#DOK`-Texte, danach alle `#SPEI`-Bereiche, danach alle `#SPUR`-Spuren, jeweils in Dateireihenfolge.

3.3 Testauftrag aus einem Assemblerlauf

Eine Trommeldatei von Hand zu schreiben ist wegen der oktalen Adressen mühsam. Für Programme in Assemblersprache übernimmt das der Assembler `c82ass`: Er wandelt ein mit symbolischen Adressen geschriebenes Programm in Oktalbefehle um. Von den erzeugten Dateien wird nur `<name>.test` benötigt; sie enthält die Trommeldatei im Zeilenformat von 3.1, jede Zeile mit der zugehörigen Quelltextzeile als Bemerkung, dazu Platzhalter für Testmarken und Tastenfolge.

```
2007: 00002006241 # beginn: 241 i ; I:=0
2010: 00002003210 # schleife: 210 azv ; ac:=(azv+i)
```



Abbildung 3: Vom Quelltext zum Testauftrag. Von Hand ergänzt werden nur die Debug-Anweisungen, die Startbedingungen und die Prüfungen.

1. Im Assembler-Quelltext an der Stelle, an der ausgewertet werden soll, einen Leerbefehl 164 mit einer vierstelligen Debug-Nummer als Adresse vorsehen, z. B. `halt: 164 0001`. Die Nummer muss zu den Testmarken `#DOK-`/`#SPEI-`/`#SPUR-` des Testauftrags passen.
2. `c82ass.sh -p <Projekt> -s <name>` aufrufen.
3. `<name>.test` als Testauftrag `Tests/asm_NNNN_<name>.txt` ablegen und die Platzhalter ergänzen: die Testmarken (`#DOK`, `#SPEI`, `#PROT STUFE=n`), die Tastenfolge, ggf. den Leerbefehl 164 und optional den Abschnitt `Prüfungen:`.

Der Assembler meldet am Ende seines Laufs, welche Dateien er erzeugt hat (echte Ausgabe von `c82ass` v0.2 für `bsp_5_16`, Auszug vom 05.10.2026):

```
C8205 Assembler C82aas Version 0.2
Pass 0
Verarbeite 'c82ass.makros', Erzeuge Macroersetzungen

Pass 1
Löse Literale auf

Pass 2
Löse Symbole auf nach Projekt_1/bsp_5.16/bsp_5_16.log und Projekt_1/bsp_5.16/bsp_5_16.sym

Pass 3
Erzeuge Kennzeichen-Befehle in Objektdatei Projekt_1/bsp_5.16/bsp_5_16.obj
…
Ausgabe als Minimalprogramm-Objektdatei nach: Projekt_1/bsp_5.16/bsp_5_16.obj
Ausgabe der formatierten Speicherbelegung nach: Projekt_1/bsp_5.16/bsp_5_16.dmp
Ausgabe der Testdatei (Spalten 1,3,4,5 fuer K1=00/01) nach: Projekt_1/bsp_5.16/bsp_5_16.test
```

Beispiel eines fertigen Testauftrags (gekürzt; Quelltext `Assembler/Projekt_1/bsp_5.16/bsp_5_16.asm`, Test `Tests/asm_0001_bsp_5_16.txt`):

```
Trommeldatei:
#DOK-0001 Test eines mit Assembler assemblierten Programmes
#SPEI-0001 2027-2033
#SPEI-0001 2100-2104
#PROT STUFE=2
0000: 00002000000 # 000 go
2000: 00002007000 # go: 000 beginn
... (weitere Zeilen unverändert aus bsp_5_16.test)
2025: 00000001164 # halt: 164 0001 ;DEBUG Nr. 0001 aktivieren
2026: 00000000020 # 020 0 ;halt
...

Tastenfolge:
H G R
```

Im Beispiel startet `H G R` bei Adresse 0000; dort steht ein Sprung zum Programmanfang. Der echte Lauf dieses Testauftrags endet mit `Test wahr` (Rückkehrcode 0):

```
$ python3 c8205_interp.py --MODUS=TEST Tests/asm_0001_bsp_5_16.txt
=== Test asm_0001_bsp_5_16 (#PROT STUFE=2) ===
…
Ende: Unbedingter Stop (US:=1) | Knoten: 1632 | BZ=2027 AC=00000005000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=1
    V2=1
…
Ergebnispruefung:
Test wahr
```

Die Knotenzahl der Endzeile ist fassungsabhängig (Hinweis in 8.2).

3.4 Geräte-Zeilen --leser1, --leser2, --stanzer1, --sw-ausgabe und --k0

Der Testauftrag kann seine Geräte selbst mitbringen: die Lochstreifen für Leser 1 und 2, die Ausgabe des Stanzers, die Schreibwerk-Ausgabe (Kanal K2) und die K0-Eingabequelle. Dazu steht je eine Zeile in der Testdatei, an beliebiger Stelle, üblicherweise bei den Testmarken im Teil `Trommeldatei:`. Anders als die Testmarken beginnen diese Zeilen nicht mit `#`, sondern wie die gleichnamigen Schalter des Python-Interpreters mit `--`. Ein Kommentar `# …` am Zeilenende wird ignoriert.

Zeile	Bedeutung	Hinweise
`--leser1 PFAD`	Lochstreifendatei des Lesers 1 (Kanal K1)	Datei oder Ordner; ein Ordner wird in Namensreihenfolge gelesen, für einen Test ist eine einzelne Datei sicherer; höchstens einmal
`--leser2 PFAD`	Lochstreifendatei des Lesers 2 (Kanal K2)	wie `--leser1`
`--stanzer1 PFAD`	Ausgabedatei bzw. Ausgabeordner des Stanzers (Kanal K1)	Datei `stanzer-ausgabe.txt`; nur bei tatsächlicher Ausgabe angelegt; höchstens einmal
`--sw-ausgabe PFAD`	jede Ausgabe auf K2 geht in die Datei `PFAD/sw-ausgabe.txt` statt ans Terminal	die Datei wird nur bei tatsächlicher Ausgabe angelegt und später angehängt; WRZL schließt die aktuelle Zeile ab; höchstens einmal
`--k0 datei=PFAD`	die Eingaben für K0 (T und SW) kommen aus Datei oder Ordner statt vom Terminal	ein Zeichen oder Name je Zeile; höchstens einmal; der Pfad muss vorhanden sein

- **Vorrang:** Ist derselbe Schalter beim Aufruf angegeben, gilt die Kommandozeile; die Zeile der Testdatei wird übergangen.
- **Relative Pfade** gelten wie auf der Kommandozeile vom aktuellen Ordner des Aufrufs aus, nicht vom Ordner der Testdatei.
- **Formale Prüfung:** Zeile ohne Wert, doppelte Zeile, `--k0` ohne `datei=` und ein nicht vorhandener `--k0`-, `--leser1`- oder `--leser2`-Pfad lassen den Test nicht starten.
- **Claude-Interpreter:** Jede dieser Zeilen gilt als virtuelle Zuordnung bei der Testdefinition.

```
Trommeldatei:
#DOK-0001 Unterprogrammtechnik, Sprung mit Rückkehrabsicht, 025
--sw-ausgabe SW-Ausgabe
#SPEI-0001 2012-2027
#PROT STUFE=2
...
```

Der Lauf dieses Tests (`Handbuch-Tests/sw-zeile.txt`: drei Unterprogramme geben je ein Zeichen aus) schreibt die Zeichen A B C nach `SW-Ausgabe/sw-ausgabe.txt`; am Terminal erscheint nur die Diagnose, keine Schreibwerk-Zeichen:

```
$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/sw-zeile.txt
=== Test sw-zeile (#PROT STUFE=2) ===
-- Diagnose --
#DOK-0001 Unterprogrammtechnik, Sprung mit Rückkehrabsicht, 025
#SPEI-0001 2012-2027
…
… (Speicherzeilen 2012 bis 2027) …
…
Ende: Unbedingter Stop (US:=1) | Knoten: 473 | BZ=2014 AC=00002012000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
…
Test wahr

$ cat SW-Ausgabe/sw-ausgabe.txt
abc
```

Hinweis Die Ausgabe ist `ABC` oder `abc`: Die Zeichenlage KB/GB des Schreibwerks wird beim Start zufällig gesetzt. Wer die Ausgabe mit `--pruefe-sw-ausgabe` prüfen will, setzt die Lage im Programm selbst (Befehl `Um. KB` bzw. `Um. GB`; Beispiel mit echtem Lauf in der Bedienungsanleitung, Abschnitt 3.6).

Hinweis Alle fünf Zeilenformen sind im Interpreter umgesetzt (Änderung 04.10.2026 (4)); im Lauf `lb-001` (9.1) wird `--leser1` als Zeile verwendet. Die frühere Abweichung zwischen Regelwerk und Interpreter ist damit beseitigt.

3.5 Die Trommel als persistente Datei (Python-Interpreter)

„Trommeldatei“ steht im Projekt für zwei verschiedene Dinge, die nicht zu verwechseln sind: den **Abschnitt** `Trommeldatei:` **einer Testdatei** (die `Adresse: Wert`-Zeilen und Testmarken des Tests; hier auch „Trommelabschnitt“) und die **Datei** `trommel.txt` des Python-Interpreters (Vorgabe `Trommel/trommel.txt`, Schalter `--trommel`): der ganze Speicher mit 4096 Zeilen. Das Schlüsselwort `Trommeldatei:` in der Testdatei bleibt unverändert.

Regeln für den Python-Interpreter (Einzelheiten, Meldungen und Schalter: Bedienungsanleitung bzw. `Python_Interpreter.md`, Abschnitt 8.5):

1. **Dauerhaft.** Die Trommel wird nicht je Test neu aufgebaut, sondern behält ihren Inhalt über Tests und Aufrufe hinweg. Alle Speicherzugriffe der Befehle (Lesen und Schreiben) laufen über sie; jede geschriebene Zelle wird in der Datei nachgeführt (spätestens am Ende des Aufrufs).
2. **Zeilenformat der Datei:** genau 4096 Zeilen `aaaa: ttttttttttt`, Adressen lückenlos aufsteigend `0000` bis `7777`, keine Kommentar- und Leerzeilen (anders als im Trommelabschnitt der Testdatei).
3. **Übernahme der Testzeilen:** Vor dem Start des Tests schreibt der Interpreter die `Adresse: Wert`-Zeilen des Trommelabschnitts auf die Trommel (Überlagerung). Eine Zelle, die der Test nennt, hat danach genau diesen Wert; eine Zelle, die er nicht nennt, behält den Wert, den sie vorher hatte. Register und Flags starten dagegen bei jedem Test neu (Kapitel 4).
4. **Formale Prüfung:** Die Trommeldatei wird einmal je Aufruf vor dem ersten Test geprüft (Kapitel 6). Ist sie fehlerhaft oder nicht vorhanden, startet kein Test des Aufrufs.
5. **Zurücksetzen:** Der Interpreter setzt die Trommel nie zurück und legt die Datei nie an. Eine leere Trommel (alle Zellen `00000000000`) erzeugt das Hilfsskript `erzeuge_leere_trommel.py`; es überschreibt eine vorhandene Datei.

```
0000: 00003000211
0001: 00003001210
0002: 00000000020
0003: 00000000000
…
3000: 00000000100
3001: 00000000045
3002: 00000000000
…
7777: 00000000000
```

Auszug aus `trommel.txt` nach einem Lauf von bspA (Kapitel 8): 4096 Zeilen, 18 Byte je Zeile. Die vom Test geschriebenen Zellen 0000 bis 0002 und 3000, 3001 stehen in der Datei, alles übrige (z. B. 3002) ist 0.

Folgen für die Testdefinition

- Das Ergebnis eines Tests hängt vom Trommelstand vor dem Test ab, sobald der Test eine Zelle **liest oder prüft** (Befehle, `#SPEI`/`#SPUR`, Speicherprüfzeilen in `Prüfungen:`), die er nicht selbst im Trommelabschnitt belegt. Solche Tests sind nur auf leerer Trommel reproduzierbar.
- **Empfehlung:** Ein Test nennt alle Zellen, auf die sich sein Ergebnis stützt, auch die mit Wert 0 (`aaaa: 00000000000`). Dann ist er von der Vorgeschichte unabhängig. Wo das nicht gewollt ist, wird vor dem Lauf eine leere Trommel erzeugt.
- **Reihenfolge:** Bei mehreren Tests auf derselben Trommel (mehrere Testdateien in einem Aufruf, mehrere Aufrufe nacheinander, Sammelläufe wie `compute_all_tests.sh`) bestimmt die Reihenfolge den Anfangszustand jedes Tests.
- **Sollwerte** (Kapitel 5 und Regressionsläufe) gelten für einen Lauf auf leerer Trommel oder für einen Test, der seine Zellen selbst nennt.
- **Protokoll:** Der Abschnitt „Ausgangsbedingungen“ (8.1) zeigt nur die vom Test geschriebenen Zeilen, nicht den ganzen Trommelinhalt vor dem Test.

Echtes Beispiel: `bsp_seite_42_025-sw-ausgabe` prüft die Zellen 2024 bis 2027 auf 0, ohne sie zu belegen. Auf leerer Trommel ist er `Test wahr` (Rückkehrcode 0); nach `bsp_seite_40`, das 2024 bis 2026 belegt, ist er `Test falsch` (Rückkehrcode 0):

```
$ python3 erzeuge_leere_trommel.py
$ python3 c8205_interp.py --MODUS=TEST Tests/bsp_seite_42_025-sw-ausgabe.test
…
Test wahr                                  # Rückkehrcode 0

$ python3 erzeuge_leere_trommel.py
$ python3 c8205_interp.py --MODUS=TEST Tests/bsp_seite_40.test
$ python3 c8205_interp.py --MODUS=TEST Tests/bsp_seite_42_025-sw-ausgabe.test
…

…
Test falsch                                 # Rückkehrcode 0
```

Claude-Interpreter. Er hat keine Trommeldatei. Er legt nur die Zeilen des Trommelabschnitts an; Zellen, die der Test nicht nennt, gelten als 0, entsprechend einem Python-Lauf auf leerer Trommel. Für die Gegenprobe (1.1) wird der Python-Lauf deshalb auf einer frisch erzeugten, leeren Trommel gestartet.

3.6 Anfangswerte der Register: #AC= und #BZ=

Zwei optionale Testmarken geben dem Akkumulator und dem Befehlszähler einen Anfangswert vor, bevor die Tastenfolge wirkt. Sie stehen im Abschnitt `Trommeldatei:`, sind voneinander unabhängig (eine, beide oder keine) und dürfen jeweils **höchstens einmal** vorkommen. Zwischen `=` und dem Wert dürfen Leerzeichen stehen; ein Kommentar `# …` hinter der Marke ist erlaubt.

```
#AC=<Oktalzahl mit 11 Ziffern>
#BZ=<Oktalzahl mit 4 Ziffern>
```

Marke	Wert	Wirkung
#AC=	11 Oktalziffern (33 Bit, die Maschinenwortbreite wie bei einem Trommelwert)	Anfangswert des Akkumulators
#BZ=	4 Oktalziffern (12 Bit, wie eine Adresse)	Anfangswert des Befehlszählers

- **Zeitpunkt:** Die Register werden als Anfangszustand des Tests **vor der ersten Taste und vor jedem Befehl** gesetzt. Die Tastenfolge (Kapitel 4) wirkt danach auf diesen Zustand.
- **Fehlt eine Marke,** startet das Register wie bisher mit 0 (`AC` = 00000000000, `BZ` = 0000). AR, OR, Flags und die übrigen Register bleiben 0.
- **Weitere Tasten:** `E` unter gehaltenem `M` rechnet vom aktuellen BZ aus weiter (BZ := BZ + 1); der AC wird dabei wie in 4.3 beschrieben überschrieben.

> **Achtung** Die Taste `G` setzt weder BZ noch AC, auch wenn `#BZ=` angegeben ist: Sie stellt nur das Befehlsregister auf „0000 000“ auf (Sprung nach 0000). Erst die Ausführung dieses Sprungs durch `R` oder durch `E` ohne `M` setzt BZ := 0000; folgt auf `G` dagegen `M+ E … E M-`, findet der Sprung nicht statt. `#BZ=` bleibt also nach `G` bis dahin erhalten und wirkt sonst vor allem bei Tastenfolgen ohne `G` (z. B. `H M+ E M- R`) und bei `H G Ü E`, das den vorgegebenen AC nach 7777 transportiert. `#AC=` bleibt von `G` ohnehin unberührt.

Echtes Beispiel `Tests/ac-001_AC_vorbelegt.txt` (die `#DOK`-Texte sind gekürzt): Der AC startet mit 100 (oktal), die Addition ohne Löschen ergibt 145 (Rückkehrcode 0):

```
Trommeldatei:
#DOK-0001 …
#DOK-0002 …
#PROT STUFE=2
#AC=00000000100
0000: 00003000210 # Einzel-Addition ohne Löschen, Zelle 3000
0001: 00000000020 # Stopp
3000: 00000000045

Tastenfolge:
H G R

Prüfungen:
BZ=0002 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0

$ python3 c8205_interp.py --MODUS=TEST Tests/ac-001_AC_vorbelegt.txt
=== Test ac-001_AC_vorbelegt (#PROT STUFE=2) ===
…
Ende: Unbedingter Stop (US:=1) | Knoten: 41 | BZ=0002 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
…
Test wahr
```

Und `Tests/acbz-002_BZ_vorbelegt_M_E.txt`: `#BZ=0010` und eine Tastenfolge ohne `G`. Das `E` unter `M` zählt BZ auf 0011, `R` entnimmt den ersten Befehl der Zelle 0011 (Stopp), daher endet der Lauf mit BZ=0012 (Rückkehrcode 0):

```
Trommeldatei:
#DOK-0001 …
#DOK-0002 …
#PROT STUFE=2
#BZ=0010
0011: 00000000020 # Stopp

Tastenfolge:
H M-gedrückt-halten E M-loslassen R

Prüfungen:
BZ=0012 AC=00000012000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0

$ python3 c8205_interp.py --MODUS=TEST Tests/acbz-002_BZ_vorbelegt_M_E.txt
=== Test acbz-002_BZ_vorbelegt_M_E (#PROT STUFE=2) ===
…
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0012 AC=00000012000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
…
Test wahr
```

Weitere Tests: `ac-001`, `bz-001_BZ_vorbelegt_G` (`#BZ=` zusammen mit `G`), `acbz-001`, `acbz-002` und `acbz-003_AC_Ue_E_Transport` in `Tests/`.

4 Schritt 2: die Tastenfolge

Nach der Trommeldatei folgt eine Tastenfolge. Sie wird auf die Ebene von Registern (mit ihren Bits) und Flags (0/1) heruntergebrochen und legt so den Ausgangszustand des Rechners vor dem Eintritt in Konnektor B vollständig fest.

> **Hinweis** Prinzip: Tastenfolgen werden nicht nur als Rohschalterzustände betrachtet, sondern bis zur Wirkung der dadurch geladenen oder ausgelösten Instruktion vorausinterpretiert. Dadurch kann der Automat ab Konnektor B im regulären Vier-Phasen-Zyklus laufen, ohne dass der manuelle Eingriff selbst im Knotengraphen als Sonderfall behandelt werden müsste. Beispiel: G stellt die Instruktion 0000 000 (Sprung nach 0000) auf; BZ wird erst 0000, wenn dieser Sprung ausgeführt wird (durch R oder E ohne M, ⟨BZ⟩ := a, ANWEISUNG-007), nicht schon durch G selbst.

4.1 Beispiel A: Ansprung des Speicherplatzes 0000 (direkt)

Trommeldatei wie in Abschnitt 3.1, Tastenfolge `H G R`:

Schritt	Aktion	Wirkung auf Register und Flags
1	Schalter H-R → „H“	Flag H := 1
2	Taste „G“	alle Flags außer H, T und SP werden gelöscht (auch BS), danach US := 1; BR := „0000 000“ (unbedingter Sprung nach Speicherplatz 0000, AR := 0000, OR := 000); K0 angewählt, SW nicht gedrückt → Flag T := 1; **BZ und AC bleiben unverändert** (G hat keinen Einfluss auf den Befehlszähler). ⟨BZ⟩ := 0 ist erst das Ergebnis der Ausführung des von G aufgestellten Befehls „0000 000“ (ANWEISUNG-007, ⟨BZ⟩ := a, hier a = 0000), also des Sprungs nach 0000. Dieser Sprung wird ausgeführt, wenn auf G kein `M+ E … E M-` folgt, hier durch `R` (Schritt 3). Der Interpreter setzt BZ := 0 deshalb erst bei `R` bzw. bei `E` ohne `M`, nicht bei G. Gibt der Test `#BZ=` vor (3.6), bleibt dieser Wert nach G bis zur Ausführung des Sprungs erhalten
3	Schalter H-R → „R“	Flag H := 0, US := 0; der von G aufgestellte BR „0000 000“ wird ausgeführt: Sprung nach 0000, ⟨BZ⟩ := 0000; die Interpretation setzt bei Konnektor B fort, mit BZ = 0000

4.2 Beispiel B: Ansprung eines beliebigen Speicherplatzes (hier 0004)

```
# Beginn Programmbefehle
0004: 00003000211
0005: 00003001210
0006: 00000000020
# Beginn Datenbereich
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G M-gedrückt-halten E E E E M-loslassen R
```

Schritt	Aktion	Wirkung auf Register und Flags
1	H-R → „H“	Flag H := 1
2	Taste „G“	alle Flags außer H, T, SP := 0 (auch BS); BR := „0000 000“; K0 angewählt, T := 1; BZ und AC bleiben unverändert (der Sprung nach 0000 des aufgestellten BR findet hier nicht statt, weil `M+ E … E M-` folgt, 4.1)
3	„M“ gedrückt halten	OR := OR ∨ 025 → BR := „0000 025“ (Sprung mit Rückkehrabsicht, Adresse 0000)
4	1. „E“	Befehlsentnahme (Phase II) durch gehaltenes M verhindert; BZ := 0001, AR := 0001; ⟨AC⟩ := 0000 0002 000 (Rücksprungadresse BZ+1 im Adressfeld)
5	2. „E“	BZ := 0002, AR := 0002; ⟨AC⟩ := 0000 0003 000
6	3. „E“	BZ := 0003, AR := 0003; ⟨AC⟩ := 0000 0004 000
7	4. „E“	BZ := 0004, AR := 0004; ⟨AC⟩ := 0000 0005 000
8	„M“ loslassen	die OR-Verknüpfung mit 025 entfällt für Folgeschritte
9	H-R → „R“	Flag H := 0; Interpretation bei Konnektor B mit BZ = 0004 → der nächste Befehl wird aus Adresse 0004 entnommen

Hinweis Das Original beschreibt bei diesem Fall als ersten Schritt „Schalter H-R → R“. Das war vermutlich ein Tippfehler: G, M und E sind nur bei Stellung „H“ wirksam. Die Tastenfolge verwendet deshalb „H“; im Bedienpult-Dokument (Abschnitt 4.4) wurde das am 02.10.2026 ebenfalls korrigiert.

Das echte Protokoll (Stufe 2) dieses Beispiels (`Handbuch-Tests/bspB.txt`) zeigt dieselbe Wirkung. Die Zeilen der Tasten sind vom Interpreter formuliert; der AC steht oktal, elf Stellen:

```
Tastenfolge: H G M-gedrückt-halten E E E E M-loslassen R
  H  : Schalter H-R -> "H", Flag H := 1
  G  : alle Flags ausser H,T,BS,SP := 0 (BS, SP: eingerastete Schalter); BR := 0000 000 (AR:=0000, OR:=000); US :=
    1; K0 angewaehlt, SW nicht gedrueckt -> T := 1; BZ bleibt unveraendert (BZ := 0 erst bei Ausfuehrung des
    Sprungs nach 0000)
  M gedrueckt halten: OR := OR ODER 025 (BR = AR 025)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0001, AR := 0001, AC := 00000002000 (BZ+1 im Adressfeld);
    BR-Adressteil := AR
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0002, AR := 0002, AC := 00000003000 (BZ+1 im Adressfeld);
    BR-Adressteil := AR
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0003, AR := 0003, AC := 00000004000 (BZ+1 im Adressfeld);
    BR-Adressteil := AR
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0004, AR := 0004, AC := 00000005000 (BZ+1 im Adressfeld);
    BR-Adressteil := AR
  M loslassen: OR-Verknuepfung mit 025 entfaellt fuer Folgeschritte
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B
  Nicht spezifizierte Register (AC, ...) starten mit 0.
```

Hinweis Der Protokollauszug stammt aus `Handbuch-Tests/bspB.txt` mit dem Stopp-Befehl in Zelle 0006. Die Trommeldatei in „Durchführung eines Tests“ (3.2) nennt für den Stopp die Zelle 0002 mit dem Wert `00000004020`; ein Programm ab 0004 liefe damit nicht zu einem Stopp, sondern über Zelle 0006 (auf leerer Trommel = 0, Sprung nach 0000) in eine Endlosschleife.

4.3 Festlegungen zu den Tasten M, Ü, E, ST, BS, SP

Taste	Festlegung
E unter gehaltenem M	Jedes E führt Phase IV (BR = ⟨AR⟩ 025: Sprung mit Rückkehrabsicht mit Löschen) und Phase I aus; Phase II ist gesperrt. Nach dem n-ten E (Start mit BZ = b): BZ := b+n, AR := BZ, und der AC enthält im Adressfeld (z_{12}–z_{23}) den Wert BZ+1, übrige Stellen 0 (das Löschbit von 025 ersetzt den AC, es wird nicht aufsummiert). Danach steht der Automat vor Phase II: ein E (nach Loslassen von M) beginnt bei Phase II (Knoten 8), ein R bei Konnektor B.
M	M hat nur zwei Stellungen: „gedrückt“ (M+) und „losgelassen“ (M-). Das Token M allein = M+ und sofort M- (Antippen). Bei M+: OR := OR ∨ 025. Wurde unter M kein E betätigt (Antippen), bleibt BR = „⟨AR⟩ 025“ aufgestellt und wird erst durch E oder R ausgeführt. Wurde mindestens ein E betätigt, entfällt der Beitrag 025 beim Loslassen; übrige Anteile im OR, z. B. von Ü, bleiben. Ist der aufgestellte BR ein Sprung mit Rückkehrabsicht (025, 027, 125, 127), beginnt die Ausführung bei Konnektor 4 (Knoten 54), nicht bei Knoten 19 (4.4).
Ü	nur bei gesetztem H wirksam. AR := 7777 (überschreibt unbedingt), OR := OR ∨ 240; BR = „7777 240“ aufgestellt, noch nicht ausgeführt. Ausgeführt durch E oder R (beides beginnt bei Knoten 19, Ende der Phase-III-Arbeit). Entsteht BR aus dem Operationsteil eines Stopp-Befehls (02Z, Z<4), ergibt 02Z ∨ 240 nicht Transport, sondern 26Z (Konjunktion mit negiertem AC): „7777 26Z“. Nur wenn OR vorher 000 war (z. B. nach G), entsteht 240.
ST	einzige Taste, die nicht an H gebunden ist, dafür elektronisch verriegelt: nur wirksam bei Bedingtem Stopp (BS) oder Unbedingtem Stopp (US). OR := OR ∨ 240; AR unverändert; US := 0; BR aufgestellt (Start bei Knoten 19 durch E oder R). Nach einem Stopp-Befehl (02Z, 06Z mit Z<4) entsteht dadurch ebenfalls 26Z, nach G (OR = 000) dagegen 240. US entsteht in einer Tastenfolge durch G und wird durch ST oder R gelöscht; ohne vorheriges G steht der Rechner zu Beginn nicht im Stopp.
BS, SP	einrastende Schalter. SP=1/SP=0 setzen das Flag SP auf 1 bzw. 0; mehr geschieht nicht. BS=1/BS=0 legen nur die Schalterstellung BS fest, das Flag BS wird erst 1, wenn ein bedingter Stopp wirksam wird. Da die Schalter mechanisch einrasten, lässt G sie unverändert (wie H); das Flag BS löscht G. Nicht an H gebunden und in jeder Reihenfolge zu den anderen Tasten zulässig.
E (ohne M)	Phase IV und I, dann Phase II und III, danach Halt (Halt-Punkt = Eintritt in Knoten 19). Nur bei gesetztem H wirksam.
Ende der Folge	Eine Tastenfolge darf ohne R enden; dann bleibt der Rechner im Halt, und das Ergebnis ist der Zustand nach dem letzten Schritt.

4.4 Wo der Lauf nach „R“ beginnt

Zustand nach der Tastenfolge	Startpunkt
kein Befehl in BR aufgestellt (z. B. `H G R`)	Konnektor B (Knoten 1) mit dem Wert in BZ; Phase I lädt den Befehl
BR aufgestellt, noch nicht ausgeführt (nach Ü, ST, M ohne E oder nach E ohne M)	Knoten 19 (Ende der Phase-III-Arbeit); Phase IV führt BR aus
BR von M ist ein Sprung mit Rückkehrabsicht (025, 027, 125, 127)	Konnektor 4 (Knoten 54): Phase III gilt als abgelaufen; nur die Resultatbehandlung (Linksverschiebung des AC) wirkt, danach wird der nächste Befehl aus der Zelle entnommen, die AR nennt (Knotenfolge 54, 55, 59, 61, 62, 63, 7, 6, 5, 8)
Tastenfolge endet im Halt (kein R)	es wird nichts mehr ausgeführt; Ergebnis = Zustand nach dem letzten Token

Zum Lesen der dritte Fall als echter Protokollauszug (`Tests/tk-001_US_H_M_R.txt`, Tastenfolge `H G M R`, Zelle 0000 enthält den Stopp 020 mit Adresse 0005; aus dem Protokoll der Stufe 2 mit `--knotentest`):

```
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B bzw. bei Knoten 19 (nach
    M mit Sprung mit Rueckkehrabsicht bei Knoten 54), wenn BR aufgestellt ist
  Nicht spezifizierte Register (AC, ...) starten mit 0.

== 2. Zustaende des Knotenautomaten ==
K054 KON 4 BZ=0000 AR=0000 AC=00000000000
K055 TST 020 BZ=0000 AR=0000 AC=00000000000 -> falsch
K059 TST 036 BZ=0000 AR=0000 AC=00000000000 -> wahr
K061 PHA Phase IV BZ=0000 AR=0000 AC=00000000000
K062 ANW 022 BZ=0000 AR=0000 AC=00000000000
K063 KON 1 BZ=0000 AR=0000 AC=00000000000
  BR=0000 005  (vor Knoten 007)  BZ=0000 AC=00000000000 FÜ=0 V1=0 V2=0 V3=0
K006 ANW 004 BZ=0000 AR=0000 AC=00000000000
K005 TST 003 BZ=0000 AR=0000 AC=00000000000 -> wahr
K008 PHA Phase II BZ=0000 AR=0000 AC=00000000000
…
```

Der Lauf beginnt bei `K054 KON 4`, nicht bei Konnektor B. Die Zeile `BR=0000 005` zeigt den Operationsteil nach der Umstellung auf den Stopp-Befehl; danach wird die Zelle 0000 über Phase II gelesen (BZ = 0001, AR = 0005).

4.5 Weitere typische Bedienfolgen (Fälle 1 bis 6)

Die folgenden Folgen sind unabhängig voneinander: Jede führt nur die Wirkung ihrer eigenen, tatsächlich aufgeführten Tasten aus. Es gibt keinen gemeinsamen Vor-Zustand; nicht genannte Register und Flags bleiben unberührt. Sie dienen der Illustration, sie gehören nicht zum Teststart.

Fall	Folge	Wirkung
1	`H G`	Flags außer H, T, SP := 0 (damit auch FÜ, FE, BS gelöscht), US := 1; BR := „0000 000“; K0, T := 1; BZ unverändert. Ohne R bleibt der Automat im Halt – das Programm ist unterbrochen.
2	`ST`	nur im Stopp (BS oder US) wirksam: OR := OR ∨ 240, AR bleibt, US := 0; BR aufgestellt, Start bei Knoten 19 durch E oder R. H bleibt unberührt.
3	`H Ü R`	Ü wirkt, weil H gesetzt ist: AR := 7777, BR := „7777 (OR ∨ 240)“. R: H := 0, Fortsetzung bei Phase IV → Ausführung von BR.
4	`H M R`	M (Antippen): OR := OR ∨ 025 → BR := „⟨AR⟩ (OR ∨ 025)“; kein E unter M, BR bleibt aufgestellt. R: bei Sprung mit Rückkehrabsicht (025, 027) Fortsetzung bei Konnektor 4 (Knoten 54), sonst bei Knoten 19.
5	`H G Ü E`	Ergebnis: ⟨7777⟩ := ⟨AC⟩ (AC in Zelle 7777 transportiert). E (einzelner Tipp): Phase IV führt „7777 240“ aus, dann Phase I, II, III; danach Halt (H bleibt 1) mit neu geladenem BR aus Zelle ⟨BZ⟩.
6	`H G M+ E M- E`	Ergebnis: ⟨AC⟩ := ⟨BZ⟩+2. Das erste E (unter M) führt Phase IV aus; Phase II bleibt gesperrt. Nach M-Loslassen führt das zweite E Phase II und III aus; danach Halt. Zusatzregel: Mehrfaches erstes E erhöht die im AC abgelegte Adresse je um 1 (zweimal: ⟨BZ⟩+3, und Zelle 0002 wird als nächster Befehl aufgestellt).

Übertragung auf BS, FÜ und FE: BS verhält sich wie US, alle vier Fälle sind anwendbar; zusätzlich beseitigt auch das Lösen des Schalters BS=0 allein den Stopp (der Ablauf geht dann wie bei nicht erfüllter Schalterbedingung weiter). FÜ: nur Fall 1; dazu ein eigener Fall „H; E gedrückt halten; G; E loslassen“, der nur den Überlauf löscht statt aller Flags und BR. FE: nur durch G aufhebbar (Fall 1).

> **Achtung** Die Zuordnung „US/BS wird übergangen“ aus dem Bedienpult-Dokument ist keine aus der Tastenfolge ableitbare Tatsache, sondern gilt nur unter der Zusatzannahme, dass AR/OR zufällig gerade den US-Befehl enthalten. Das gehört zur Anwendung, nicht zur Definition der Sequenzen. Ob die Hardware nach „H, G“ wirklich US = 1 anzeigt, ist in der Originaldokumentation nicht belegt und bleibt offen; wirkungslos ist es in jedem Fall, weil „R“ US unabhängig von seinem Wert überwindet.

4.6 Das Makro #STARTE

`#STARTE n` ist eine Kurzschreibweise für die Tastenfolge zum **Ansprung eines beliebigen Speicherplatzes n** (Beispiel B in 4.2):

```
#STARTE n      ≙      H G M+ E E … E M- R        (E genau n-mal)
```

Regel	Festlegung
Wert	`n` ist eine Oktalzahl mit 1 bis 4 Ziffern (0 bis 7777, eine Speicheradresse) und darf nicht leer sein. `n` ist zugleich die Zahl der Anschläge `E`: `#STARTE 12` löst 10 (dezimal) Anschläge aus und spricht den Speicherplatz `0012` an.
Stellung	an beliebiger Stelle der Testdatei außer im Abschnitt `Prüfungen:`, üblicherweise bei den Testmarken im Abschnitt `Trommeldatei:`; Kommentar `# …` am Zeilenende erlaubt; höchstens einmal je Test.
Ersatz der Tastenfolge	`#STARTE` ersetzt die Tastenfolge vollständig. Der Abschnitt `Tastenfolge:` darf dann fehlen oder leer sein; enthält er Tasten, ist das ein Formfehler.
Wirkung	genau die der ausgeschriebenen Folge (4.3): `G` stellt BR auf „0000 000“, `M+` verknüpft den OR mit 025, jedes `E` zählt BZ um 1 weiter. Nach dem n-ten `E` gilt BZ = n, AR = n und im AC steht im Adressfeld n+1. `R` setzt bei Konnektor B mit BZ = n fort; der erste Befehl kommt aus der Zelle n.
Grenzfall `#STARTE 0`	kein `E`, die Folge lautet `H G M+ M- R`, also ein Antippen von `M` ohne `E`. Das ist kein bloßer Start bei 0000, sondern ein aufgestellter Sprung mit Rückkehrabsicht nach 0000 (4.4); für einen schlichten Start bei 0000 dient `H G R`.

Regel	Festlegung
Protokoll	Im Abschnitt „Ausgangsbedingungen“ steht `#STARTE nnnn` statt der ausgeschriebenen Tasten, danach die Wirkung der erzeugten Tasten in **kompakter Form**: Folgen von mindestens vier `E` unter gehaltenem `M` erscheinen als die ersten zwei `E`-Zeilen, eine Sammelzeile `... k weitere E` und die letzte `E`-Zeile (bei `#STARTE 1000` sind das 4 statt 512 Zeilen). Gekürzt wird nur das Protokoll, nicht die Rechnung. Eine ausgeschriebene Tastenfolge wird dagegen **nie** gekürzt.
Zusammenspiel	verträglich mit `#AC=`, `#BZ=` und `#INTERRUPT`; `#BZ=` bleibt wie bei `H G …` bis zum ersten `E` unter `M` erhalten. Als Argument von `#INTERRUPT` ist das Makro nicht vorgesehen (dort wird die Tastenfolge ausgeschrieben).

Echtes Beispiel `Tests/st-001_STARTE_4.txt` (gleichwertig zu `Tastenfolge: H G M+ E E E E M- R`; die `#DOK`-Texte sind gekürzt):

```
Trommeldatei:
#DOK-0001 …
#STARTE 4
#PROT STUFE=2
0004: 00003000211 # 211 Laden
0005: 00003001210 # 210 Addieren
0006: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045

Prüfungen:
BZ=0007 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Im Protokoll (Stufe 2) stehen die erzeugten Tasten in kompakter Form; die vier `E` erscheinen als zwei `E`-Zeilen, eine Sammelzeile und die letzte `E`-Zeile:

```
  #STARTE 0004 (Makro: H G M+ 4x E M- R, E 4-mal = dezimal)
Tastenfolge: (aus #STARTE erzeugt)
  H  : Schalter H-R -> "H", Flag H := 1
  G  : alle Flags ausser H,T,BS,SP := 0 (BS, SP: eingerastete Schalter); BR := 0000 000 (AR:=0000, OR:=000); US :=
    1; K0 angewaehlt, SW nicht gedrueckt -> T := 1; BZ bleibt unveraendert (BZ := 0 erst bei Ausfuehrung des
    Sprungs nach 0000)
  M gedrueckt halten: OR := OR ODER 025 (BR = AR 025)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0001, AR := 0001, AC := 00000002000 (BZ+1 im Adressfeld);
    BR-Adressteil := AR
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0002, AR := 0002, AC := 00000003000 (BZ+1 im Adressfeld);
    BR-Adressteil := AR
  ... 1 weitere E (BZ 0003 bis 0003): je Anschlag BZ := BZ+1, AR := BZ, AC := BZ+1 im Adressfeld (kompakte
    Protokollform bei #STARTE)
  E  : (M gehalten) Befehlsentnahme gesperrt, BZ := 0004, AR := 0004, AC := 00000005000 (BZ+1 im Adressfeld);
    BR-Adressteil := AR
  M loslassen: OR-Verknuepfung mit 025 entfaellt fuer Folgeschritte
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B
```

Der Lauf endet mit `Test wahr` (Rückkehrcode 0). Die Gegenprobe `Tests/st-002_STARTE_4_ausgeschrieben.txt` mit der ausgeschriebenen Tastenfolge liefert denselben Endzustand (Rückkehrcode 0); `st-003_STARTE_oktal_12` zeigt die oktale Zählung (`#STARTE 12` spricht Zelle 0012 an).

Formfehler: Ein Wert, der keine Oktalzahl mit 1 bis 4 Ziffern ist (hier `#STARTE 19`), lässt den Test nicht starten (Rückkehrcode 16):

```
Trommeldatei:
#STARTE 19
0000: 00000000020

Tastenfolge:
H G R

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/starte-formfehler.txt
=== Test starte-formfehler (#PROT STUFE=1) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 2: #STARTE '19' ungueltig (1 bis 4 Oktalziffern erwartet)
```

4.7 Unterbrechung des Programmlaufs: #INTERRUPT

Ein Test kann den Ablauf der Befehlsschleife an bestimmten Stellen unterbrechen und dabei eine **weitere Tastenfolge** bedienen. Das bildet Eingriffe des Bedieners mitten im Programm nach (Halt, Register ändern, fortsetzen). Wie die übrigen Testmarken hat `#INTERRUPT` keine Entsprechung in der Hardware; die Knotentabelle bleibt unverändert.

```
#INTERRUPT  H <weitere Tasten>
```

Regel	Festlegung
Schreibweise	Tasten-Token wie in der Tastenfolge, durch Leerzeichen getrennt; gehaltene Taste `+`, Loslassen `-` (z. B. `#INTERRUPT H G M+ E M- R`). **Die erste Taste muss `H` sein.** Kommentar `# …` am Zeilenende erlaubt.
Stellung	im Abschnitt `Trommeldatei:` und **nur zwischen zwei Befehlszeilen** (`Adresse: Wert`). Die Zeile gehört zur unmittelbar davor stehenden Befehlszeile mit der Adresse p; vor der ersten und hinter der letzten Befehlszeile ist sie unzulässig. Sie ist keine Speicherzelle und kein Sprungziel.
Wann sie wirkt	nach dem Befehl der Zeile davor, vor Phase I des nächsten Befehls (Zustand nach Phase IV). Das gilt nur, wenn der Ablauf **sequentiell** weiterläuft, also nach dem Befehl auf p der BZ den Wert p+1 hat (auch bei einem nicht erfüllten bedingten Sprung). Springt der Befehl woandershin, wird die Zeile **übersprungen**. Bei jedem erneuten Erreichen (Schleife) wirkt sie von Neuem. Nicht ausgelöst wird sie nach einem Befehl, der nicht aus der Zelle p stammt (z. B. durch `E` oder `R` aus einem aufgestellten BR ausgeführt), und nach einem Stopp.
Was die Tasten bewirken	Sie wirken wie in 4.3 auf den Zustand an der Unterbrechungsstelle, nicht auf den Anfangszustand des Tests: Register und Flags bleiben, wie der letzte Befehl sie hinterlassen hat. Ohne vorheriges `G` ist OR der Operationsteil des letzten Befehls: `H Ü E` ergibt nach einem Ladebefehl 211 die Operation 251, nicht 240; für den reinen Transport nach 7777 ist `H G Ü E` nötig.
Fortsetzung	Ist die **letzte Taste** `R` **oder** `ST`, geht der Lauf weiter: bei Konnektor B mit dem von den Tasten hinterlassenen BZ, bei aufgestelltem BR wie in 4.4. BZ und AC gelten so, wie die Tasten sie verlassen haben. `G` setzt BZ nicht, stellt aber BR auf „0000 000“: Führt ein folgendes `R` diesen Sprung aus, beginnt der Lauf bei 0000 von vorn; folgt `M+ E … E M-`, entfällt der Sprung. Bei `ST` als letzter Taste wird H gelöscht (ST wirkt nur bei US oder BS).
Ende	**Jede andere letzte Taste** beendet den Test an dieser Stelle (Rechner im Halt, H gesetzt). Das Ergebnis ist der Zustand nach der letzten Taste; ein Abschnitt `Prüfungen:` gilt für diesen Zustand. Spätere `#INTERRUPT`-Zeilen kommen dann nicht mehr zum Zug.
Mehrfach	mehrfach je Testdatei, aber höchstens einmal je Befehlszeile (je Adresse p).
Protokoll	In „Ausgangsbedingungen“ steht je Zeile `#INTERRUPT nach Befehl pppp: Tasten`; im Ablauf eine Zeile `--- INTERRUPT nach Befehl pppp ...` mit BZ und AC, die Wirkung jeder Taste und `--- Ende INTERRUPT ...` bei Fortsetzung.

Echtes Beispiel 1, Fortsetzung (`Tests/int-001_weiter.txt`): Nach dem Laden (Zelle 0000) wird unterbrochen, `H R` setzt den Lauf fort; das Ergebnis ist das gleiche wie ohne Unterbrechung (Rückkehrcode 0).

```
Trommeldatei:
#PROT STUFE=2
0000: 00003000211 # 211 Laden
#INTERRUPT H R
0001: 00003001210 # 210 Addieren
0002: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G R

Prüfungen:
BZ=0003 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Aus dem Protokoll (Stufe 2, mit `--knotentest`): die Zeile `#INTERRUPT nach Befehl 0000: H R` unter den Ausgangsbedingungen und der Ablauf an der Unterbrechungsstelle:

```
  #INTERRUPT nach Befehl 0000: H R
…
--- INTERRUPT nach Befehl 0000 (Testzeile 4): BZ=0001 AC=00000000100 ---
  H  : Schalter H-R -> "H", Flag H := 1
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B
--- Ende INTERRUPT: Ablauf wird fortgesetzt ---
```

Echtes Beispiel 2, Ende (`Tests/int-003_Ue_E.txt`, Rückkehrcode 0): Die letzte Taste ist `E`, nicht `R`; der Test endet an der Unterbrechungsstelle. Der AC wurde nach 7777 transportiert; die `Prüfungen:` gelten für diesen Zustand:

```
Trommeldatei:
#PROT STUFE=2
0000: 00003000211 # 211 Laden
#INTERRUPT H G Ü E # Transport AC nach 7777
0001: 00003001210 # 210 Addieren
0002: 00000000020 # 020 Stopp
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G R

Prüfungen:
7777: 00000000100

$ python3 c8205_interp.py --MODUS=TEST Tests/int-003_Ue_E.txt
=== Test int-003_Ue_E (#PROT STUFE=2) ===
…
Ende: Halt (Schalter H): INTERRUPT nach Befehl 0000 endet ohne R/ST | Knoten: 27 | BZ=0002 AC=00000000100 AR=3001
    | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
…
Ergebnispruefung:
  7777: soll=00000000100 ist=00000000100 OK
…
Test wahr
```

In `Tests/` liegen dazu `int-001_weiter` und `int-003_Ue_E`.

5 Schritt 3 (optional): Prüfungen

Nach der Tastenfolge kann ein dritter Abschnitt `Prüfungen:` folgen. Fehlt er, endet der Test ohne automatische Ergebnisprüfung. Ist er vorhanden, vergleicht der Interpreter nach der interpretativen Abarbeitung den Endzustand mit den Sollwerten und vermerkt `Test wahr` oder `Test falsch`. Dadurch wird jeder Testlauf zum Regressionstest: Ein neuer Lauf wird gegen die zuletzt bestätigten Sollwerte verglichen, ohne dass ein Mensch das Protokoll erneut lesen muss.

5.1 Aufbau

Teil	Inhalt
1. Register-/Flag-Zeile (höchstens eine)	Zuweisungen `SCHLÜSSEL=WERT`, durch Leerzeichen getrennt, beliebige Reihenfolge. Schlüssel: `BZ`, `AC`, `AR`, `FE`, `FUE` (für FÜ), `BS`, `US`, `E`, `A`, `V1`, `V2` – genau die Teilmenge, die auch die Endzeile des Interpreters ausgibt. Nicht vorgesehen: OR, H, SP, T, SW, K1–K3. Die Zeile muss nicht vollständig sein; jeder Schlüssel höchstens einmal. `BZ=` und `AR=`: 4 Oktalstellen · `AC=`: 11 Oktalstellen · Flags: genau 0 oder 1.
2. Speicherprüfzeilen (beliebig viele)	je eine pro Zeile im Format `Adresse: Wert` (4 + 11 Oktalstellen): Inhalt, den die Trommeladresse nach Testende haben muss. Verglichen wird mit dem Trommelstand nach dem Lauf; bei Adressen, die der Test nicht selbst belegt, hängt der Istwert vom Trommelstand vor dem Test ab (3.5). Dieselbe Adresse darf nicht mit widersprüchlichem Sollwert auftreten.
3. Dateiprüfungen (höchstens zwei)	je eine Zeile `--pruefe-stanzer1 DATEI` bzw. `--pruefe-sw-ausgabe DATEI` (Abschnitt 5.4).

Fehlt die Register-/Flag-Zeile, beginnt der Abschnitt unmittelbar mit Speicherprüfzeilen. Ein leerer Abschnitt `Prüfungen:` gilt wie ein fehlender; ein Abschnitt, der nur Dateiprüfungen enthält, ist gültig.

```
Prüfungen:
BZ=0004 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
3000: 00000000100
3001: 00000000045
```

5.2 Auswertung und Ausgabe

- Die Prüfung erfolgt erst nach Abschluss der Abarbeitung, gegen den dann erreichten Endzustand.
- Jede angegebene Zuweisung und jede Speicherzeile wird einzeln verglichen; zu jeder wird Soll und Ist ausgegeben, damit im Fehlerfall sofort sichtbar ist, welche Prüfung abweicht.
- `Test wahr` gibt es nur, wenn alle Einzelprüfungen stimmen (einschließlich der Dateiprüfungen nach 5.4); sobald mindestens eine abweicht, ist das Ergebnis `Test falsch`.
- Die Ausgabe erfolgt stufenunabhängig auf dem Bildschirm (wie die Diagnose zu Befehl 164); bei Protokollstufe 2 zusätzlich am Ende der Protokolldatei (Abschnitt `== 3. Ergebnisprüfung ==`), im Anschluss an die Endzeile.
- **Test ohne ordnungsgemäßes Ende:** Bricht die Abarbeitung vorher ab (Stub, Laufzeitfehler, Knotenlimit), wird keine Prüfung vorgenommen und stattdessen vermerkt, dass sie entfällt (`Ergebnisprüfung: nicht durchgeführt (Test nicht ordnungsgemäß beendet)`). Ein solcher Fall gilt nicht automatisch als `Test falsch`.

Beispiel: derselbe Lauf, einmal mit richtigem und einmal mit falschem Sollwert (`BZ=0003` statt `BZ=0004`; `Handbuch-Tests/pruef-wahr.txt` und `pruef-falsch.txt`):

```
$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/pruef-wahr.txt
=== Test pruef-wahr (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=0004 ist=0004 OK
  AC: soll=00000000145 ist=00000000145 OK
  AR: soll=0000 ist=0000 OK
  FE: soll=0 ist=0 OK
  FUE: soll=0 ist=0 OK
  BS: soll=0 ist=0 OK
  US: soll=1 ist=1 OK
  E: soll=0 ist=0 OK
  A: soll=0 ist=0 OK
  V1: soll=0 ist=0 OK
  V2: soll=0 ist=0 OK
  3000: soll=00000000100 ist=00000000100 OK
  3001: soll=00000000045 ist=00000000045 OK
Test wahr

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/pruef-falsch.txt
=== Test pruef-falsch (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=0003 ist=0004 FEHLER
  AC: soll=00000000145 ist=00000000145 OK
… (übrige Zeilen wie oben, alle OK)
Test falsch
```

Der Interpreter meldet das Ergebnis auch als Rückkehrcode an das Betriebssystem; er bezieht sich immer auf genau einen Test:

Code	Bedeutung
0	keine Vorab-Prüfungs-Fehler; `Prüfungen:`-Abschnitt und/oder Dateiprüfung definiert und Ergebnis `Test wahr`
4	keine Vorab-Prüfungs-Fehler; weder `Prüfungen:`-Abschnitt noch Dateiprüfung angegeben
6	Prüfung definiert, aber Ergebnis `Test falsch` (auch: Soll- oder Ist-Datei fehlt, Dateiinhalt verschieden); bleibt auch bei `Prüfungen: False` 6 (5.5)
8	sonstige Laufzeitfehler (Stub, Gerät nicht nutzbar, Abbruch mit Strg+C)
16	Vorab-Prüfung negativ (Formfehler, ungültige Tastenfolge, ungültiger `Prüfungen:`-Abschnitt, fehlender Geräte-Pfad, Testdatei nicht gefunden) oder Trommeldatei fehlerhaft, nicht vorhanden bzw. nicht beschreibbar (3.5)
2	ungültiger Aufruf (unbekannter Schalter, fehlende Regelwerk-Dateien); der Interpreter startet keinen Test

Im Beispiel liefern `pruef-wahr` den Code 0 und `pruef-falsch` den Code 6; ein Test ohne `Prüfungen:` (etwa `spur`, Kapitel 7) endet mit 4.

5.3 Typische Sollwerte

Wenn der Test …	prüfen Sie
mit einem Stopp-Befehl endet	`US=1`, `BZ=` (um 1 hinter dem Stopp-Befehl), `AC=` mit dem erwarteten Ergebnis
Zahlen in den Speicher schreibt	die Zielzellen als Speicherprüfzeilen (Zellen, die der Test selbst belegt, oder Lauf auf leerer Trommel, 3.5); Vorsicht bei Gruppenbefehlen: sie überschreiben bis zu 32 Zellen einer Spur
einen Überlauf erwartet	`FUE=1`
eine Eingabe liest	`E=0`, `FE=0` und den AC; mit leerem Leser: `E=1`
Vergleichersprünge benutzt	`V1=`, `V2=` (V3 = beide 1)
auf Stanzer oder Schreibwerk ausgibt	die Dateiprüfung nach 5.4

5.4 Dateiprüfungen für Stanzer und Schreibwerk

Programme, die auf Kanal K1 (Lochstreifenstanzer) oder K2 (Schreibwerk) ausgeben, erzeugen eine Ausgabedatei (`stanzer-ausgabe.txt` bzw. `sw-ausgabe.txt`). Zwei weitere, ebenfalls optionale Prüfungen vergleichen diese Dateien mit je einer Soll-Datei. Sie stehen als Zeile im Abschnitt `Prüfungen:` der Testdatei (je Zeile eine Prüfung, jede höchstens einmal); der Python-Interpreter nimmt sie zusätzlich als Schalter der Kommandozeile entgegen, und ein Schalter hat Vorrang vor der Zeile der Testdatei. Beim Claude-Interpreter gilt die entsprechende virtuelle Zuordnung bei der Testdefinition.

Zeile / Schalter	Soll-Datei wird verglichen mit
`--pruefe-stanzer1 DATEI`	der Datei, die der Lauf über `--stanzer1 PFAD` (Schalter oder Zeile der Testdatei, 3.4) erzeugt hat (bei einem Verzeichnis: `stanzer-ausgabe.txt` darin)
`--pruefe-sw-ausgabe DATEI`	der Datei, die der Lauf über `--sw-ausgabe PFAD` (Schalter oder Zeile der Testdatei, 3.4) erzeugt hat (bei einem Verzeichnis: `sw-ausgabe.txt` darin)

Beide Prüfungen können einzeln oder gemeinsam angegeben werden, auch zusammen mit Register-, Flag- und Speicherprüfungen.

- **Soll-Datei:** die hinter dem Schalter genannte DATEI. Ein reiner Dateiname ohne Verzeichnis liegt im Verzeichnis der Testdatei (üblicherweise `Tests/`); eine Angabe mit Verzeichnis gilt, wie sie geschrieben ist. In diesem Projekt liegen die Soll-Dateien in `Test-Ergebnisvergleiche/` und werden mit diesem Pfad genannt (`--pruefe-sw-ausgabe Test-Ergebnisvergleiche/<Name>_erwartet.txt`, Pfad relativ zum Arbeitsverzeichnis).
- **Namenskonvention:** Der Name einer Soll-Datei endet auf `-ausgabe.txt` oder `_erwartet.txt`; `compute_all_tests.sh` überspringt solche Dateien. Ein anderer Name lässt das Skript die Soll-Datei als Test starten (Rückkehrcode 16, weil Tastenfolge und Zeilenformat fehlen).
- **Ist-Datei:** die vom Lauf erzeugte Ausgabedatei. Maßgeblich ist nur eine Datei, die dieser Lauf angelegt hat; eine übrig gebliebene Datei aus einem früheren Lauf gilt als nicht vorhanden, wenn der Lauf selbst keine Ausgabe auf dem Gerät vorgenommen hat.
- **Prüfung falsch:** Soll-Datei fehlt, Ist-Datei fehlt (auch beide) oder der Inhalt unterscheidet sich. Verglichen wird zeilenweise wie bei `diff`; Unterschiede nur im Zeilenende (CR LF gegen LF) oder ein fehlendes Zeilenende nach der letzten Zeile zählen nicht, Leerzeichen am Zeilenende dagegen schon. **Prüfung wahr:** beide Dateien vorhanden und zeilenweise gleich.
- **Gesamtergebnis:** `Test wahr` nur, wenn jede Einzelprüfung nach 5.1 und jede Dateiprüfung wahr ist. Wie bei 5.2 entfallen die Dateiprüfungen, wenn der Test nicht ordnungsgemäß endet.
- **Ausgabe:** je Dateiprüfung eine Zeile mit Soll- und Ist-Pfad und `OK` bzw. `FEHLER` samt Grund (Soll-Datei fehlt, Ist-Datei fehlt, vom Lauf nicht angelegt, Inhalt verschieden); bei verschiedenem Inhalt folgt der `diff` (Soll gegen Ist, höchstens 100 Zeilen).
- **Formale Prüfung:** `--pruefe-stanzer1` ist nur zusammen mit `--stanzer1`, `--pruefe-sw-ausgabe` nur zusammen mit `--sw-ausgabe` zulässig (Schalter oder Zeile der Testdatei), weil sonst keine Ist-Datei existieren kann. Als Schalter der Kommandozeile sind beide nur bei einer Testdatei je Aufruf zulässig. Eine fehlende Soll-Datei ist kein Formfehler, sondern das Prüfergebnis falsch.

Beispiel (echter Lauf, `Tests/bsp_seite_42_025_stanzer.test`): Das Programm wählt K1 an (4600 030) und stanzt drei Zeichen; die Soll-Datei `Test-Ergebnisvergleiche/bsp_seite_42_025_stanzer-stanzer-ausgabe.txt` enthält die Zeilen `00110001`, `00110010` und `00100011`. Ende der Testdatei:

```
Prüfungen:
BZ=2014 AC=00002012000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
2012: 00000001164
...
2022: 00000000777
--pruefe-stanzer1 Test-Ergebnisvergleiche/bsp_seite_42_025_stanzer-stanzer-ausgabe.txt
```

```
$ python3 c8205_interp.py --MODUS=TEST --stanzer1 Stanzer1 Tests/bsp_seite_42_025_stanzer.test
=== Test bsp_seite_42_025_stanzer (#PROT STUFE=2) ===
-- Diagnose --
#DOK-0001 Unterprogrammtechnik,  Sprung mit Rückkehrabsicht, 025
#SPEI-0001 2012-2027
…
… (Speicherzeilen 2012 bis 2027) …
…
Ende: Unbedingter Stop (US:=1) | Knoten: 473 | BZ=2014 AC=00002012000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
…
Ergebnispruefung:
  BZ: soll=2014 ist=2014 OK
… (übrige Zeilen alle OK)
  Stanzer 1: soll=Test-Ergebnisvergleiche/bsp_seite_42_025_stanzer-stanzer-ausgabe.txt
    ist=Stanzer1/stanzer-ausgabe.txt OK
Test wahr

$ cat Stanzer1/stanzer-ausgabe.txt
00110001
00110010
00100011
```

Rückkehrcode 0 bei `Test wahr`, 6 bei `Test falsch`.

5.5 Erwartetes Ergebnis „falsch“: Prüfungen: False

Manche Tests sollen bewusst `Test falsch` ergeben (Negativtests, die zeigen, dass eine Abweichung erkannt wird). Der Rückkehrcode 6 würde in den Sammelläufen als Fehler gemeldet. Deshalb darf hinter dem Abschnittskopf **optional** `False` stehen:

```
Prüfungen: False
BZ=0003 AC=00000000046 AR=0003
3001: 00000000045
```

Regel	Festlegung
Schreibweise	Die Kopfzeile lautet `Prüfungen:`, danach höchstens Leerzeichen und das Wort `False` (Groß- und Kleinschreibung unerheblich), sonst nichts. Der übrige Abschnitt (Register-/Flag-Zeile, Speicherprüfzeilen, Dateiprüfungen) ist unverändert und wird wie sonst ausgewertet und protokolliert. Jeder andere Text hinter `Prüfungen:` (auch `True`) ist ein Formfehler (Rückkehrcode 16).
Wirkung im Interpreter	Die Auswertung ändert sich nicht: Gesamtergebnis (`Test wahr` / `Test falsch`) und Rückkehrcode (0 bzw. 6) bleiben, wie sie sind. Der Interpreter ergänzt nur eine Vermerkzeile hinter dem Gesamtergebnis (Bildschirm, bei `#PROT STUFE=2` auch Protokolldatei): `Erwartetes Ergebnis: falsch (Prüfungen: False)`.
Wirkung in den Skripten	Die Skripte erkennen die Kopfzeile in der Testdatei (Muster `^Prüfungen:[[:space:]]*[Ff][Aa][Ll][Ss][Ee][[:space:]]*$`; ohne Ordner wird auch in `Tests/` gesucht). Der Rückkehrcode bleibt 6, der Lauf gilt als **Test wahr** und macht keine Ausgabe: `c8205_bis_test.sh` startet den eigentlichen Test mit `--NO-TERMOUT` (das Laden des Minimalprogramms ist nicht betroffen); `test.sh` und `test_bis.sh` übernehmen die Regel über `c8205_bis_test.sh`; `compute_all_tests.sh` meldet Code 6 bei einer solchen Testdatei nicht (wie 0 und 4).
Grenzen	Alle anderen Rückkehrcodes (2, 8, 16) werden wie bisher behandelt und von `compute_all_tests.sh` gemeldet. Ein Test mit `Prüfungen: False` ohne jede Prüfzeile hat keine Ergebnisprüfung (Code 4); `False` bleibt ohne Wirkung. Endet der Test nicht ordnungsgemäß, entfällt die Ergebnisprüfung samt Vermerk.

Echter Lauf: `Handbuch-Tests/pruef-falsch.txt` (5.2, Sollwert `BZ=0003` statt `BZ=0004`) mit der Kopfzeile `Prüfungen: False`, als Kopie `Handbuch-Tests/pruef-erwartet-falsch.txt`. Ende der Testdatei:

```
Prüfungen: False
BZ=0003 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
3000: 00000000100
3001: 00000000045
```

Mit der aktuellen Fassung des Interpreters (0.95-001) endet derselbe Lauf so (Rückkehrcode 16):

```
$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/pruef-erwartet-falsch.txt
=== Test pruef-erwartet-falsch (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 14: Pruefungen: ungueltiges Element 'False'
  - Zeile 15: Pruefungen: ungueltige Zeile: 'BZ=0003 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0'
```

Achtung Abweichung Regelwerk ↔ Interpreter (Fassung 0.95-001): Die Kopfzeile `Prüfungen: False` steht im Regelwerk (Ergänzung 10.10.2026) und wird von den Skripten erkannt; der Python-Interpreter dieser Fassung meldet sie dagegen als Formfehler und startet den Test nicht. Wer `Prüfungen: False` verwenden will, braucht eine Interpreterfassung, die den Vermerk „Erwartetes Ergebnis: falsch (Prüfungen: False)“ ausgibt.

6 Formale Gültigkeitsprüfung vor dem Start

Vor jedem Test werden Trommeldatei, Tastenfolge, Prüfungen und Testmarken formal geprüft. Der Test startet nur, wenn die Prüfung insgesamt positiv verläuft. Bei einer Unstimmigkeit darf er nicht eigenmächtig korrigiert und gestartet werden; die Durchführung wartet auf eine Klärung mit dem Auftraggeber.

Bereich	Geprüft wird
Trommeldatei (Abschnitt der Testdatei)	Zeilenformat (Adresse 4, Wert 11 Oktalstellen, nur Oktalzahlen, zulässige Kommentar- und Leerzeilen); keine Adresse mit widersprüchlichem Wert mehrfach belegt
Persistente Trommel (Python-Interpreter)	Die Datei `trommel.txt` muss vorhanden und beschreibbar sein und genau 4096 Zeilen `aaaa: ttttttttttt` mit lückenlos aufsteigenden Adressen `0000` bis `7777` enthalten. Die Prüfung gilt für den ganzen Aufruf und läuft einmal vor dem ersten Test; schlägt sie fehl, startet kein Test (3.5)
`#SPEI-xxxx`	nach Regelwerk: nur zu einem im selben Test definierten `#DOK-xxxx` gleicher Nummer; ssss-eeee bildet eine gültige Spanne
`#PROT STUFE=n`	n ist 0, 1 oder 2; höchstens einmal; Fehlen ist gültig (Standard: 1)
`#AC=`, `#BZ=`	jede höchstens einmal; der Wert besteht aus genau 11 (`#AC=`) bzw. 4 (`#BZ=`) Oktalziffern; Fehlen ist gültig (das Register startet mit 0, 3.6)
`#STARTE`	Wert nicht leer, 1 bis 4 Oktalziffern, höchstens einmal, nicht im Abschnitt `Prüfungen:`, keine zusätzliche Tastenfolge; ein Test mit `#STARTE` braucht keinen Abschnitt `Tastenfolge:` (4.6)
`#INTERRUPT`	nur im Abschnitt `Trommeldatei:`, zwischen zwei Befehlszeilen, höchstens einmal je Befehlszeile; mindestens eine Taste, erste Taste `H`, nur bekannte Tasten, zulässige M-Folge (4.3)
Tastenfolge	ergibt einen vollständig definierten Ausgangszustand. Ungültig: M- ohne vorheriges M+, zweites M+ bei gedrücktem M, R oder Ende bei gedrücktem M, unbekannte Tasten, Ende weder mit R noch im Halt. Gültig, aber ohne Wirkung: Ü und E bei nicht gesetztem H.
Geräte-Zeilen	`--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe`, `--k0`: Zeile mit Wert; jede höchstens einmal; `--k0` in der Form `datei=PFAD`; die Pfade von `--k0`, `--leser1` und `--leser2` müssen vorhanden sein (3.4). Eine Zeile `--SW-TERMINAL` ist stets ein Formfehler (nur Kommandozeile, 9.2)
Prüfungen:	unbekannte Schlüssel, falsche Stellenzahl oder unzulässiger Wert, mehrfach genannter Schlüssel, Zeile außerhalb des Adresse:Wert-Formats, widersprüchlich mehrfach belegte Prüfadresse; hinter `Prüfungen:` ist nur `False` zulässig (5.5)
Dateiprüfungen	`--pruefe-stanzer1` setzt `--stanzer1`, `--pruefe-sw-ausgabe` setzt `--sw-ausgabe` voraus; höchstens eine Zeile je Prüfung; als Schalter nur bei einer Testdatei je Aufruf (5.4)

Beispiel 1: Eine Testdatei mit drei Fehlern in der Trommeldatei (`Handbuch-Tests/formfehler.txt`). Der Interpreter nennt alle Zeilenfehler auf einmal, startet nicht und liefert den Rückkehrcode 16:

```
Trommeldatei:
#PROT STUFE=3
000: 00003000211
0001: 3000210
Tastenfolge:
H G
M-loslassen R

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/formfehler.txt
=== Test formfehler (#PROT STUFE=1) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 2: #PROT STUFE=3 ungueltig (0,1,2)
  - Zeile 3: ungueltiges Zeilenformat: '000: 00003000211'
  - Zeile 4: ungueltiges Zeilenformat: '0001: 3000210'
Rückkehrcode: 16
```

Die Tastenfolge dieser Datei ist ebenfalls ungültig (`M-loslassen` ohne vorheriges `M-gedrückt-halten`). Sie wird erst bewertet, wenn die Trommeldatei formal in Ordnung ist; nach Korrektur der Zeilen meldet der Interpreter sie als nächstes:

```
Trommeldatei:
#PROT STUFE=0
0000: 00000000020
Tastenfolge:
H G M- R

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/tastenfehler.txt
=== Test tastenfehler (#PROT STUFE=0) ===
TEST NICHT ORDNUNGSGEMAESS BEENDET: Tastenfolge nicht gueltig/definiert
Ende: Tastenfolge nicht gueltig/definiert | Knoten: 0 | BZ=0000 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=0 E=0
    A=0 V1=0 V2=0
Rückkehrcode: 16
```

Ebenso bei den neuen Marken: Fünf `#INTERRUPT`-Zeilen, die alle gegen die Stellungsregel oder gegen die erste Taste `H` verstoßen (`Handbuch-Tests/interrupt-formfehler.txt`; dieser Test gehört bewusst nicht nach `Tests/`, weil `compute_all_tests.sh` den Rückkehrcode 16 als Fehlschlag melden würde). Der Interpreter meldet eine Zeile je Fehler (Rückkehrcode 16):

```
Trommeldatei:
#INTERRUPT H R          # Fehler: vor der ersten Befehlszeile
0000: 00000000020
0001: 00000000020
#INTERRUPT M+ E         # Fehler: erste Taste nicht H, M bleibt gedrückt
#INTERRUPT X R          # Fehler: erste Taste nicht H, unbekannte Taste X
#INTERRUPT R            # Fehler: erste Taste nicht H
0002: 00000000020
#INTERRUPT H R          # Fehler: keine Befehlszeile dahinter

Tastenfolge:
H G R

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/interrupt-formfehler.txt
=== Test interrupt-formfehler (#PROT STUFE=1) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 2: #INTERRUPT: steht vor der ersten Befehlszeile (muss ZWISCHEN zwei Befehlszeilen stehen)
  - Zeile 5: #INTERRUPT: die erste Taste muss H sein, gefunden 'M+'
  - Zeile 5: #INTERRUPT: Tastenfolge endet mit gedrueckter Taste M
  - Zeile 6: #INTERRUPT: die erste Taste muss H sein, gefunden 'X'
  - Zeile 6: #INTERRUPT: unbekannte Taste 'X'
  - Zeile 7: #INTERRUPT: die erste Taste muss H sein, gefunden 'R'
  - Zeile 9: #INTERRUPT: steht nicht zwischen zwei Befehlszeilen (keine Zeile danach)
```

Auch `--SW-TERMINAL` als Zeile der Testdatei ist ein Formfehler; der Schalter gilt nur auf der Kommandozeile (9.2, Rückkehrcode 16):

```
Trommeldatei:
--SW-TERMINAL
0000: 00000000020

Tastenfolge:
H G R

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/swterm-zeile.txt
=== Test swterm-zeile (#PROT STUFE=1) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 2: --SW-TERMINAL ist nur als Kommandozeilenschalter zulaessig, nicht in der Testdatei
```

Auch ein Aufruf mit nicht vorhandener Testdatei meldet das laut und startet nichts (Rückkehrcode 16):

```
$ python3 c8205_interp.py --MODUS=TEST Tests/gibtsnicht.txt
FEHLER: Testdatei nicht gefunden: 'Tests/gibtsnicht.txt' (aktuelles Verzeichnis: …/c8205)
```

Beispiel 2: Die Trommeldatei selbst ist fehlerhaft (hier in einer Kopie `trommel_kaputt.txt`: in Zeile 3 fehlt der Doppelpunkt). Der Interpreter prüft sie einmal je Aufruf, nennt die Fehler (höchstens 50) und führt keinen Test aus (Rückkehrcode 16). Abhilfe: leere Trommel mit `erzeuge_leere_trommel.py` neu erzeugen (3.5).

```
$ python3 c8205_interp.py --MODUS=TEST --trommel trommel_kaputt.txt Handbuch-Tests/vorlage.txt
FEHLER: Trommeldatei "trommel_kaputt.txt" ist formal fehlerhaft - Test(s) starten nicht:
  - Zeile 3: ungueltiges Format (erwartet "aaaa: tttttttttttt"): '0002 00000000000'
Rückkehrcode: 16
```

Achtung Abweichung Regelwerk ↔ Interpreter (Fassung 07.10.2026): Das Regelwerk (Durchführung eines Tests, Abschnitt 5) lässt `#SPEI` nur zu einem `#DOK` gleicher Nummer zu. Der Python-Interpreter prüft das nicht; er gibt die Zellen auch ohne `#DOK` aus (Interpreter-Dokument: `#SPEI` und `#SPUR` brauchen kein `#DOK`). Ob das Regelwerk oder der Interpreter anzupassen ist, ist nicht entschieden.

Echter Lauf zu dieser Abweichung (`Handbuch-Tests/spei-ohne-dok.txt`: Befehl 164 mit Adresse 0007, `#SPEI-0007`, aber kein `#DOK-0007`; Rückkehrcode 4, weil kein `Prüfungen:`-Abschnitt vorhanden ist):

```
Trommeldatei:
#SPEI-0007 0000-0001
#PROT STUFE=0
0000: 00000007164
0001: 00000000020
Tastenfolge:
H G R

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/spei-ohne-dok.txt
=== Test spei-ohne-dok (#PROT STUFE=0) ===
-- Diagnose --
#SPEI-0007 0000-0001
0000: 00000007164
0001: 00000000020
Ende: Unbedingter Stop (US:=1) | Knoten: 41 | BZ=0002 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
```

Hinweis Grundsatz „laut scheitern“ (gilt ebenso später bei der Abarbeitung des Knotengraphen): Ergibt sich eine Unklarheit, ein Widerspruch oder eine fehlende Festlegung, wird das laut gemeldet und der Test angehalten, statt eigenmächtig eine Annahme zu treffen. Das gilt unabhängig davon, ob Python-Interpreter oder KI den Test ausführen.

7 Diagnose beim Leerbefehl 164

Die Diagnoseausgabe gibt während des Laufs Zwischenstände aus. Sie ist, wie die Marken `#DOK` und `#SPEI`, eine von der Hardware abweichende Zusatzdefinition ohne Entsprechung in der Elektronik.

7.1 Auslösebedingung

Tritt in der Befehlsfolge der Befehl 164 (oktal; Leerbefehl, B = 1, A = 6, Z = 4) mit einer Adresse a auf und ist für xxxx = a eine Testmarke `#DOK-xxxx` definiert, wird zusätzlich zur regulären Abarbeitung die Diagnose ausgegeben. Der Python-Interpreter wählt zur Adresse a die zugehörigen `#DOK`-Texte, `#SPEI`-Bereiche und `#SPUR`-Spuren unabhängig voneinander aus (Abweichung in Kapitel 6); passt keine Marke, entfällt die Diagnose ganz.

7.2 Ablauf

1. Der Befehl 164 wird zuerst genau so interpretiert, wie es der Knotenautomat vorgibt; die Ausführung selbst erfährt keine Sonderbehandlung.
2. Danach wird ausgegeben: die Testmarke `#DOK-xxxx` mit ihrem Text.
3. Ist zu xxxx ein `#SPEI-xxxx ssss-eeee` definiert: die Markenzeile selbst und alle Zellen ssss bis eeee zeilenweise `Adresse: Wert`.
4. Ist ein `#SPUR-xxxx aaaa` definiert: danach, nach allen `#SPEI`-Ausgaben, die Markenzeile und alle 32 Zellen der Spur in physischer Zugriffsreihenfolge der Trommel, Format wie bei `#SPEI`.
5. Danach wird die Interpretation regulär fortgesetzt.

Die Diagnose erscheint stufenunabhängig immer auf dem Bildschirm; bei Stufe 2 zusätzlich in der Protokolldatei. Nach dem Regelwerk gehören zur Textzeile auch BZ, AC und die Flags FE, FÜ, BS, US, E, A; der Python-Interpreter gibt diese Zeile nicht aus (Änderung 29.09.2026: es stehen Marke, Text und Speicherzeilen da).

Beispiel mit `#SPUR`: Der Test (`Handbuch-Tests/spur.txt`) hält bei Befehl 164 (Adresse 0020) und gibt die Spur der Zelle 4001 aus. Die Zellen erscheinen in der Reihenfolge der Trommel, nicht aufsteigend (vgl. Tabelle in der Rechnerbeschreibung, Kapitel 6.1):

```
Trommeldatei:
#DOK-0020 x
#SPUR-0020 4001
#PROT STUFE=0
0000: 00000020164
0001: 00000000020
Tastenfolge:
H G R

$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/spur.txt
=== Test spur (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0020 x
#SPUR-0020 4001
4000: 00000000000
4015: 00000000000
4032: 00000000000
4007: 00000000000
4024: 00000000000
4001: 00000000000
4016: 00000000000
… (25 weitere Zellen, alle 0, in Trommelreihenfolge) …
Ende: Unbedingter Stop (US:=1) | Knoten: 41 | BZ=0002 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
```

Zelle für Zelle: 4000, 4015, 4032, 4007, 4024, 4001, … – das ist die Spalte „logische Sektoradresse“ der physischen Reihenfolge (Interleaving, Schritt +05 oktal).

> **Hinweis** Die Leerbefehle 164 und 166 werden ohne Resultatbehandlung ausgeführt (das Z-Feld bewirkt keine Verschiebung des AC). Es ist daher unschädlich, die Marke an eine beliebige Programmstelle zu setzen, und sie eignet sich zum Messpunkt zwischen zwei Schritten.

8 Protokoll und Protokolltiefe

#PROT STUFE=	Knotenprotokoll	Protokolldatei	Diagnose (164) und Prüfung
0	keines	keine	immer auf dem Bildschirm
1 (Standard)	auf dem Bildschirm, knapp: eine Zeile je durchlaufenem Knoten	keine	immer auf dem Bildschirm
2	in der Protokolldatei	ja (für den ganzen Test, auch ohne Ausgabebefehle)	Bildschirm und Datei

Die Stufen sind nützlich vor allem bei Programmen mit vielen Durchläufen (z. B. Schleifen über Gruppen- oder Wiederholungsbefehle), bei denen ein Protokoll jedes einzelnen Knotens unübersichtlich lang würde. Der Python-Interpreter nimmt die Knotenzeilen nur auf, wenn er mit `--knotentest` aufgerufen wird (Bedienungsanleitung, Abschnitt 6.4).

8.1 Inhalt der Protokolldatei (Stufe 2)

1. **Ausgangsbedingungen:** die vom Test übernommenen Trommelzeilen (nicht der ganze Trommelinhalt, 3.5), die Testmarken `#AC=` und `#BZ=` (falls angegeben, mit den Anfangswerten von AC und BZ, 3.6), die Zeilen `#INTERRUPT nach Befehl pppp: Tasten` (4.7) und die Tastenfolge mit ihrer Wirkung auf Register und Flags; bei `#STARTE` steht die Marke statt der ausgeschriebenen Tasten, die Wirkung der erzeugten Tasten in kompakter Form (4.6).
2. **Zustände des Knotenautomaten:** die ab Konnektor B (oder dem Startpunkt nach 4.4) erreichten Knoten in der durchlaufenen Reihenfolge, mit BZ, AR und AC; bei einer Unterbrechung mit `#INTERRUPT` dazwischen die Zeilen `--- INTERRUPT nach Befehl pppp ...` und `--- Ende INTERRUPT ...` samt Wirkung der Tasten (4.7).
3. **Debug-Ausgaben** nach Kapitel 7 (`#DOK`, `#SPEI`, `#SPUR`).
4. **Ende:** Endgrund und Endzeile; danach, falls ein `Prüfungen:`-Abschnitt vorhanden ist, die Ergebnisprüfung mit Soll und Ist je Wert (einschließlich der Dateiprüfungen nach 5.4) und dem Gesamtergebnis; bei `Prüfungen: False` folgt die Vermerkzeile `Erwartetes Ergebnis: falsch (Prüfungen: False)` (5.5).

Zum Lesen ein echter Protokollauszug (Beispiel A aus Abschnitt 4.1: Laden, Addieren, Stopp; `Handbuch-Tests/bspA.txt`, die Marke `#PROT STUFE=2` und `--knotentest` waren gesetzt):

```
$ python3 c8205_interp.py --MODUS=TEST --knotentest --NO-TERMOUT --out bspA.log Handbuch-Tests/bspA.txt
Test bspA

== 1. Ausgangsbedingungen ==
Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0):
  0000: 00003000211
  0001: 00003001210
  0002: 00000000020
  3000: 00000000100
  3001: 00000000045
  #DOK-0001 Beispiel A: Laden, Addieren, Stopp
  #PROT STUFE=2
Tastenfolge: H G R
… (Wirkung je Taste wie in 4.1)
== 2. Zustaende des Knotenautomaten ==
K001 KON B BZ=0000 AR=0000 AC=00000000000
  BR=0000 000  (vor Knoten 002)  BZ=0000 AC=00000000000 FÜ=0 V1=0 V2=0 V3=0
K002 PHA Phase I BZ=0000 AR=0000 AC=00000000000
K003 ANW 001 BZ=0001 AR=0000 AC=00000000000
K004 TST 002 BZ=0001 AR=0000 AC=00000000000 -> falsch
K005 TST 003 BZ=0001 AR=0000 AC=00000000000 -> wahr
K008 PHA Phase II BZ=0001 AR=0000 AC=00000000000
K009 ANW 005 BZ=0001 AR=0000 AC=00000000000
K010 TST 006 BZ=0001 AR=0000 AC=00000000000 -> falsch
K011 ANW 007 BZ=3000 AR=0000 AC=00000000000
K012 TST 008 BZ=3000 AR=0000 AC=00000000000 -> falsch
K015 PHA Phase III BZ=3000 AR=0000 AC=00000000000
K016 ANW 001 BZ=0001 AR=3000 AC=00000000000
K017 TST 011 BZ=0001 AR=3000 AC=00000000000 -> wahr
…
K045 ANW 030 BZ=0003 AR=0000 AC=00000000145

== Ende ==
Unbedingter Stop (US:=1)
BZ=0003 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Achtung Die Überschrift „Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0)“ im Protokoll stammt noch aus der Zeit vor der persistenten Trommel und trifft seit 07.10.2026 nicht mehr zu: Nicht genannte Zellen behalten ihren bisherigen Inhalt (3.5). Die Zeilen darunter sind nur die vom Test geschriebenen Zellen, nicht der ganze Trommelinhalt. Ob der Interpreter oder das Regelwerk angepasst wird, ist nicht entschieden.

8.2 Eine Knotenzeile lesen

Zeile	Lesart
`K001 KON B BZ=0000 AR=0000 AC=…`	Knoten 1 (Konnektor B): Eintritt in den Ablauf. Danach BZ, AR, AC oktal.
`BR=0000 000 (vor Knoten 002) …`	Befehlsregister (Adressteil und Operationsteil) vor Beginn von Phase I; dazu BZ, AC, FÜ, V1, V2, V3. Die Zeile steht auch vor Knoten 7 (Sprung über Konnektor 1 aus Knoten 63), wie im Auszug aus 4.4: `BR=0000 005 (vor Knoten 007)`.
`K002 PHA Phase I …`	Phasenzuordnung: Beginn der Phase I (analog Phase II: Knoten 8, III: 15, IV: 31/61/67)
`K003 ANW 001 BZ=0001 AR=0000 …`	Knoten 3 ist eine Anweisung, ANWEISUNG-001: AR := BZ und gleichzeitig BZ := AR + 1 mit den alten Werten. BZ=0001 ist deshalb richtig (nicht 0000+1 mit neuem AR).
`K004 TST 002 … -> falsch`	Knoten 4, ein Test (ANWEISUNG-002, „X-Abbruch-Merker gesetzt?“): Ergebnis falsch → weiter über den falsch-Ausgang
`K016 ANW 001 …`	derselbe Anweisungs-Typ in Phase III (Knoten 16): AR wird auf die Operandenadresse gestellt; BZ := alter AR + 1 = BADR + 1
`K045 ANW 030 …`	ANWEISUNG-030, unbedingter Stopp: Endknoten, US := 1
`K054 KON 4 …`	Konnektor 4 (Knoten 54): Einstieg nach M mit Sprung mit Rückkehrabsicht; Phase III gilt als abgelaufen (4.4)

Hinweis Die „Knoten“-Zahl der Endzeile und die Zahl der Knotenzeilen sind fassungsabhängig und keine Zeitangabe (alle Zeiten sind 0). Sie eignen sich nicht als Sollwert eines Tests.

9 Ein- und Ausgabe in Tests

Programme können Zeichen ein- und ausgeben. Im Test werden die Geräte nur mit den Kanal- und Adresszuordnungen des Originals nachgebildet, ohne mechanische Bauteile. Jede virtuelle Datei wird nur angelegt, wenn der Test das Gerät benutzt.

Kanal	Eingabe (Regelwerk)	Ausgabe (Regelwerk)
K0 mit T oder SW	über die Host-Tastatur: pro Eingabebefehl genau ein Zeichen wird angefordert (nicht eine Zeile); es wird nach der Kodetabelle in die Eingabeverschlüsselung umgesetzt und mit dem Adressregister ODER-verknüpft; danach wird der Eingabebefehl umgestellt	—
K1	Leser 1: virtueller Lochstreifen (Textdatei), pro Eingabebefehl eine Zeile aus acht Zeichen 0/1 (Spuren w v u p 8 4 2 1; Tr wird nicht dargestellt)	Stanzer 1: Datei, ein ausgegebenes Zeichen je Zeile, acht 0/1 in derselben Spurreihenfolge
K2	Leser 2: wie Leser 1, aber getrennte Lochstreifendatei	Schreibwerk: zeichenweise Textausgabe; WRZL beendet die Zeile (mehrere WRZL = Leerzeilen)

Kanalanwahl. Die Kanäle K0 bis K3 werden entweder durch die rastenden Pult-Tasten K0–K3 (Teil der Tastenfolge; die Taste setzt das Flag des gewählten Kanals auf 1 und die der anderen drei auf 0, bei K0 zusätzlich T oder SW je nach Schalter SW) oder durch einen Anwahlbefehl im Programm gewählt: `4700 030` = K0, `4600 030` = K1, `4500 030` = K2, `4400 030` = K3. Ein zur Laufzeit ausgeführter Anwahlbefehl **übersteuert** die zuvor per Taste eingestellte Kanalstellung. Kanal K3 bewirkt in dieser Ausbaustufe nichts (kein Gerät zugeordnet).

K0 aus Datei. Ist für K0 eine Eingabedatei oder ein Eingabeverzeichnis zugeordnet (`--k0 datei=PFAD`), kommen die Eingaben beider Subkanäle (T und SW) aus dieser Quelle statt vom Terminal; mehrere Dateien in einem Verzeichnis werden in alphanumerischer Reihenfolge verarbeitet. Ist der Pfad vorhanden, aber keine Datei (mehr) da, wartet der Interpreter still, bis eine neue Datei erscheint.

9.1 Lochstreifendateien

- Für Leser 1 und Leser 2 werden stets getrennte Dateien benutzt. Jeder Leser muss durch `--leser1` bzw. `--leser2` (Schalter oder Zeile der Testdatei, 3.4) definiert werden; fehlt die Zuordnung, ist auf dem Kanal keine Eingabe möglich.
- Ist die aktuelle Datei zu Ende und tritt für denselben Leser ein weiterer Eingabebefehl auf, wird eine neue Lochstreifendatei angefordert; der Python-Interpreter wartet dann still (ohne Meldung), bis eine neue Datei erscheint.
- Wird eine Datei nicht vollständig verbraucht, bleibt sie für den Leser geladen und wird bei einem späteren Eingabebefehl an der zuletzt entnommenen Stelle fortgesetzt.
- Geht der Rechner in den Halt (Stopp, Fehler-Halt oder sonst), werden alle angebrochenen Dateien beider Leser beendet und entfernt; ein neuer Test fordert seine Dateien neu an.
- Eine Zeile mit gerader Zahl von Einsen ist ein Fehler bei Eingabe (FE := 1, der Lauf hält). Leerzeilen sind Vorspann und werden übergangen. Das Lochbild `11111111` („Irrung Zeichen“) wird übersprungen: keine Paritätsprüfung, kein FE, nicht ins AR; der Eingabebefehl liest das nächste Zeichen (Interpreter-Dokument 8.2, Ergänzung 04.10.2026).

Echtes Beispiel `Tests/lb-001_Irrung_Zeichen_uebersprungen.txt`: Das Programm wählt K1 an und liest ein Zeichen. Der Streifen (`--leser1`-Zeile der Testdatei) enthält erst die Irrung `11111111`, dann die Ziffer 1 (`00000001`). Die Irrung wird übersprungen; der AC erhält den Inhalt der Zelle 0001 (Eingabeadresse 01 mit Adreßteil 0000 verknüpft):

```
Trommeldatei:
#DOK-0001 Lochbandeingabe: Anwahl K1, ein Eingabe-Halt, unbedingter Stopp
--leser1 Test-virtuelleGeraete/Leser1/lb-001_irrung_zeichen_dann_1.txt
#PROT STUFE=2
0000: 00002000000 # 000 go
2000: 00004600030 # 030 4600 ; Anwahl Kanal 1 (Leser 1)
2001: 00000000010 # 010 0 ; Eingabe-Halt 1. Art
2002: 00000000020 # 020 0 ; unbedingter Stopp
…

$ cat Test-virtuelleGeraete/Leser1/lb-001_irrung_zeichen_dann_1.txt
11111111
00000001

$ python3 c8205_interp.py --MODUS=TEST Tests/lb-001_Irrung_Zeichen_uebersprungen.txt
=== Test lb-001_Irrung_Zeichen_uebersprungen (#PROT STUFE=2) ===
…
Ende: Unbedingter Stop (US:=1) | Knoten: 82 | BZ=2003 AC=00000000001 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
…
Test wahr
```

Rückkehrcode 0. BZ=2003, AC=00000000001, kein Fehler bei Eingabe (FE=0).

9.2 Schreibwerk

Das Regelwerk beschreibt eine Ausgabe in die Datei `sw-ausgabe.txt` (höchstens einmal je Test angelegt, nur bei tatsächlicher Ausgabe, später angehängt statt überschrieben, am Testende zum Download bereitgestellt). Ist dem Test die Datei zugeordnet (Schalter `--sw-ausgabe PFAD` oder gleichnamige Zeile in der Testdatei, 3.4), geht jede Ausgabe auf K2 dorthin; fehlt die Zuordnung, erscheint sie auf dem Terminal (Dunkelgelb). Beides zugleich gibt es nie. Die Ausgabe ist vom Schalter SW unabhängig; ein Warten auf SW findet nicht statt.

Schalter `--SW-TERMINAL`**.** Zusätzlich zu Datei oder Terminal kann der Python-Interpreter mit `--SW-TERMINAL` aufgerufen werden: Dann öffnet er neben dem Terminal ein Ausgabefenster „SW Ausgabe“, in dem **jede** Ausgabe auf K2 zusätzlich erscheint, gleichgültig ob eine Datei zugeordnet ist; die Wirkung von `--sw-ausgabe` bleibt unverändert. Nicht angegeben bedeutet AUS (kein Fenster). Der Schalter gilt für alle Tests eines Aufrufs und ist **nur auf der Kommandozeile** zulässig, nicht als Zeile der Testdatei (6).

- Das Fenster enthält nur Text (130 Zeichen × 36 Zeilen), trägt die Überschrift „SW Ausgabe“, ist verschiebbar und erscheint standardmäßig rechts unten auf dem Desktop. Es bleibt auch nach dem Ende des Aufrufs **immer offen**, bis es mit Klick auf X geschlossen wird.
- Mit `--SW-TERMINAL` erscheint außerdem jede angenommene Eingabe über K0 (Pult-Tastatur T oder Schreibwerk SW) als Echo in Dunkelgelb im selben Fenster; die Ausgaben des Rechners stehen dort in der Standardfarbe. Die Eingaben der Lochbandleser (K1, K2) werden nicht ausgegeben.
- Text im Fenster lässt sich mit der Maus markieren und in die Zwischenablage kopieren; das Kopierverfahren wählt der Schalter `--zwischenablage auto|tk|system` (Bedienungsanleitung, Abschnitt 5.1).

> **Hinweis** Die Schreibwerk-Ausgabe über K2 ist unabhängig vom Schalter SW (Hardwarebeschreibung und Regelwerk 6.3.1); der Schalter SW betrifft nur die Eingabeseite (K0). Für den Stanzer gibt es dagegen keinen Terminal-Ausweg: ohne `--stanzer1` unterbleibt die Ausgabe auf K1 ganz.

9.3 Gegenüberstellung der Ausführenden

	Python-Interpreter	Claude-Interpreter
K0-Eingabe	Terminal (ein Tastendruck je Eingabebefehl) oder `--k0 datei=PFAD` (Schalter oder Zeile der Testdatei)	die KI fordert pro Eingabebefehl ein Zeichen auf dem Host-Bildschirm an und nimmt es entgegen
Leser 1/2	`--leser1`, `--leser2` (Datei oder Ordner; Schalter oder Zeile der Testdatei)	die KI fordert die Lochstreifendatei für den jeweiligen Leser an
Stanzer 1	`--stanzer1 PFAD` (Schalter oder Zeile der Testdatei)	Datei `stanzer-ausgabe.txt`, am Testende zum Download
Schreibwerk	`--sw-ausgabe PFAD` (Schalter oder Zeile der Testdatei) → Datei `sw-ausgabe.txt`; sonst Terminal, Dunkelgelb; zusätzlich Fenster „SW Ausgabe“ mit `--SW-TERMINAL` (nur Schalter)	Datei `sw-ausgabe.txt`, am Testende zum Download
Trommel	persistente Datei `trommel.txt` (3.5)	keine Datei; nicht genannte Zellen gelten als 0
Protokoll	`--out …`, bei Stufe 2 Datei	Datei `protokoll.txt`, am Testende zum Download

10 Den Claude-Interpreter einsetzen

10.1 Was die KI braucht

Material	Zweck
Knotengraph (`Eingänge_der_Knoten.md`) und Variablentabelle T2 (`Beschreibung_der_Knoten.md`)	der Ablauf selbst und die Bedeutung jeder ANWEISUNG
Befehlsliste (`Befehlsliste.md`)	die Formel je Operationscode (ANWEISUNG-021 verweist dorthin)
`Datenstruktur_und_Verarbeitungseinheiten.md`, `Bedienpult_mit_Schaltern.md`	Register, Felder, Flags; Tasten und Schalter
Diese Testdefinition (`Durchführung_eines_Tests.md`) und die Testdatei	Format, Tastenfolge, Marken, Protokoll, Trommel (2c)
`Magnettrommel.md` (bei `#SPUR`/`#SPEI`)	Sektorreihenfolge der Spur
`Kodetabellen.md`, `Lochstreifenleser.md` (bei Ein- und Ausgabe)	Zeichenkode; Kanäle, Leser, Stanzer, Schreibwerk
`Überblick_und_Gesamtaufbau.md` (optional)	Geräteaufbau und die vier Phasen, hilfreich bei Rückfragen
`baz-wirkungstabelle.md` (optional)	Cache bereits hergeleiteter Knotenpfade und Formeln je BAZ-Wert; der Python-Interpreter liest ihn nicht. Nur verwenden, wenn die Einträge zu den aktuellen Fassungen von Knotengraph, T2 und Befehlsliste passen, sonst neu herleiten

Faustregel: Wer sich die Auswahl sparen will, lädt alle `.md`-Dateien hoch, außer `Python_Interpreter`, `PDF`, `Projektstruktur`, `Projektueberblick` und der Assembler-Dokumentation. Das kostet Kontext, den lange Protokolle brauchen. Dazu kommen die Testdatei und gegebenenfalls Dateien für Leser und K0. Eine Trommeldatei gehört nicht dazu (3.5).

10.2 Vorgehen

Eine Test-Anforderungs-Datei (die Testdatei) anlegen: Aufbau nach Kapitel 2 bis 5, Vorlage in 12.2. Sie wird mit den Dateien aus 10.1 der KI übergeben. Start und Startanweisung: Kurzanleitung, Weg A. Für den Vergleich mit dem Python-Interpreter (1.1) wird dessen Lauf auf einer frisch erzeugten, leeren Trommel gestartet, weil die KI nicht genannte Zellen als 0 annimmt.

10.3 Was der Python-Interpreter braucht

Kapitel 10 behandelt sonst den Claude-Interpreter. Dieser Abschnitt ist das Gegenstück für Weg B der Kurzanleitung: alle Dateien und Pfade, die vorhanden sein müssen, damit `c8205_interp.py` einen Test ausführen kann. Der Interpreter liest zur Laufzeit nur **zwei** Regelwerk-Dateien; dazu kommen die Trommeldatei und die Testdatei. Alles Weitere hängt davon ab, welche Geräte der Test benutzt.

Kleinster lauffähiger Projektordner (Vorgaben der Pfade, alle relativ zu `c8205_interp.py`):

```
Projektordner/
  c8205_interp.py
  erzeuge_leere_trommel.py            nur zum Erzeugen / Zurücksetzen der Trommel
  markdown-dateien/
    C8205-Interpreter-Eingänge_der_Knoten.md       Knotengraph
    C8205-Interpreter-Beschreibung_der_Knoten.md   Variablentabelle T2
  Trommel/
    trommel.txt                       4096 Zeilen, les- und beschreibbar
  Tests/
    <Testdatei>                       oder ein beliebiger Pfad zur Testdatei
```

Immer nötig

Datei / Pfad	Zweck	Wenn es fehlt
Python 3 (`python3`)	führt das Skript aus; nur die Standardbibliothek, nichts zu installieren	Meldung des Betriebssystems
`c8205_interp.py`	der Interpreter selbst (Funktionen je ANWEISUNG, Kodetabelle 1 eingebettet)	–
`C8205-Interpreter-Eingänge_der_Knoten.md`	Knotengraph. Liegt im Verzeichnis `markdown-dateien/` neben `c8205_interp.py` (unabhängig vom aktuellen Ordner); anderer Ort: `--mdverz VERZ`	„error: Knotengraph nicht gefunden: …“, Rückkehrcode 2
`C8205-Interpreter-Beschreibung_der_Knoten.md`	Variablentabelle T2 (Bedeutung der ANWEISUNG-nnn), im selben Verzeichnis	„error: Knotenbeschreibung nicht gefunden: …“, Rückkehrcode 2
`Trommel/trommel.txt`	die persistente Trommel (3.5): genau 4096 Zeilen, les- und beschreibbar. Der Interpreter legt sie nie an; erzeugen mit `python3 erzeuge_leere_trommel.py`. Anderer Ort: `--trommel PFAD` (nur Kommandozeile)	„FEHLER: Trommeldatei … ist formal fehlerhaft“ bzw. „… kann nicht geoeffnet werden“; kein Test startet, Rückkehrcode 16
die Testdatei	der Test (Kapitel 2 bis 5). Ein Name ohne Pfad wird unterhalb des aktuellen Ordners und des Skript-Ordners gesucht (Groß-/Kleinschreibung egal); mehrere Treffer: der Test startet nicht	„FEHLER: Testdatei nicht gefunden: …“, Rückkehrcode 16

Ziele, die der Interpreter anlegt (Schreibrechte nötig):

Pfad	Inhalt
`Test-Protokolle/` im Skript-Ordner	Protokoll bei `#PROT STUFE=2`, je Test `<Testname>/protokoll.txt`; anderer Ort: `--out VERZ` (Ordner werden angelegt) oder `--out DATEI.log`. Der Wrapper schreibt `Test-Protokolle/<Testname>.log` relativ zum aktuellen Ordner
`trommel.txt`	jede Zelländerung eines Befehls wird in der Datei nachgeführt (3.5)
Ziele der Geräte-Ausgabe	siehe nächste Tabelle (Stanzer, Schreibwerk)

Nur wenn der Test das Gerät benutzt (ein nicht zugeordnetes Gerät ist kein Fehler, solange das Programm es nicht benutzt):

Gerät	Zuordnung und übliche Ablage	Wenn es fehlt
Leser 1 / Leser 2 (K1 / K2)	`--leser1 PFAD`, `--leser2 PFAD` (Schalter oder Zeile der Testdatei, 3.4); Lochstreifendatei mit je einer Zeile aus acht Zeichen 0/1; Ablage `Test-virtuelleGeraete/Leser1/`, `Leser2/`	Zuordnung fehlt, das Programm liest: rote Fehlermeldung, der Test endet (Rückkehrcode 8). Pfad nicht vorhanden: Vorab-Prüfung, Rückkehrcode 16
Eingabe K0 aus Datei	`--k0 datei=PFAD` (Schalter oder Zeile); Ordner oder Datei mit einem Zeichen oder Namen je Zeile; Ablage `Test-virtuelleGeraete/T-SW-Eingabe/`	ohne Zuordnung: Eingabe vom Terminal (interaktiv). Pfad nicht vorhanden: Vorab-Prüfung, Rückkehrcode 16
Stanzer 1 (K1)	`--stanzer1 PFAD` (Schalter oder Zeile); Ablage `Test-virtuelleGeraete/Stanzer1/`. Das Ziel wird angelegt: ein vorhandener Ordner oder ein Pfad mit `/` am Ende wird Ordner (darin `stanzer-ausgabe.txt`), ein neuer Pfad ohne `/` die Ausgabedatei selbst	Zuordnung fehlt, das Programm stanzt: rote Fehlermeldung, der Test endet (Rückkehrcode 8); es gibt keinen Terminal-Ausweg
Schreibwerk (K2)	`--sw-ausgabe PFAD` (Schalter oder Zeile): Ordner, darin `sw-ausgabe.txt`, wird angelegt; Ablage `Test-virtuelleGeraete/SW-Ausgabe/`	ohne Zuordnung: Ausgabe am Terminal (Dunkelgelb), kein Fehler
Soll-Dateien der Dateiprüfungen	`--pruefe-stanzer1 DATEI`, `--pruefe-sw-ausgabe DATEI` (5.4); ein Name ohne Ordner liegt im Verzeichnis der Testdatei (üblicherweise `Tests/`); in diesem Projekt: `Test-Ergebnisvergleiche/`, Pfad relativ zum Arbeitsverzeichnis	Soll-Datei fehlt: Prüfung falsch, `Test falsch`, Rückkehrcode 6 (kein Formfehler)

Optional, aber üblich – für den einzelnen Lauf nicht nötig:

Datei	Wozu
`erzeuge_leere_trommel.py`	erzeugt `trommel.txt` leer oder setzt sie zurück (überschreibt ohne Rückfrage); liegt neben `c8205_interp.py`
`c8205_interpreter.sh` / `c8205_interpreter.bat`	Wrapper: erstes Argument die Testdatei, Protokoll nach `Test-Protokolle/<Testname>.log`. Im Projektordner und aus ihm aufrufen: Er ruft `python3 c8205_interp.py` sowie `./markdown-dateien` und `Test-Protokolle/` relativ zum aktuellen Ordner auf
`compute_all_tests.sh`	Sammellauf (11.1); braucht den Wrapper, `Tests/`, `Test-Ergebnisvergleiche/`, `Test-virtuelleGeraete/Stanzer1/` und `erzeuge_leere_trommel.py`
`c8205_bis_test.sh`, `test_bis.sh`, `test.sh`	Tests der BIS-Bausteine (11.4): erst das Minimalprogramm laden, dann den eigentlichen Test; `test_bis.sh` und `test.sh` rufen den Wrapper für jeden Test auf
`c8205_befehlssweep.py`	Vollständigkeitstest über alle 512 Operationscodes (11.3)

Nicht nötig: alle übrigen `.md`-Dateien (Dokumentation und Regelwerk für den Claude-Interpreter; der Python-Interpreter liest nur die beiden oben genannten), `baz-wirkungstabelle.md`, die PDFs und der Assembler. Die Dateien `*-ausgabe.txt` und `*_erwartet.txt` in `Test-Ergebnisvergleiche/` sind Soll-Dateien einzelner Tests, keine Voraussetzung für den Start.

Echte Läufe mit fehlenden Voraussetzungen: ein leeres Regelwerk-Verzeichnis (hier `leer/`) und eine nicht vorhandene Trommeldatei:

```
$ python3 c8205_interp.py --MODUS=TEST --mdverz leer Tests/0015-030.txt
c8205_interp.py: error: Knotengraph nicht gefunden: leer/C8205-Interpreter-Eingänge_der_Knoten.md
Rückkehrcode: 2

$ python3 c8205_interp.py --MODUS=TEST --trommel trommel_fehlt.txt Tests/0015-030.txt
FEHLER: Trommeldatei "trommel_fehlt.txt" ist formal fehlerhaft - Test(s) starten nicht:
  - Trommeldatei nicht gefunden: trommel_fehlt.txt
Rückkehrcode: 16
```

Probelauf, wenn alles vorhanden ist (leere Trommel erzeugen, dann ein Test mit Ausgabe auf dem Terminal; erwartet wird `Jahr 1972`, `Test wahr`, Rückkehrcode 0):

```
$ python3 erzeuge_leere_trommel.py
$ python3 c8205_interp.py --MODUS=TEST Tests/0015-030.txt
=== Test 0015-030 (#PROT STUFE=2) ===

Jahr 1972
…
Ende: Unbedingter Stop (US:=1) | Knoten: 308 | BZ=7257 AC=00000002000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
…
Test wahr
Rückkehrcode: 0
```

11 Referenztests und Regression

11.1 Testbestand und Gesamtlauf

Für die Regression liegen in `Tests/` zum Stand 21.10.2026 **199 Tests** (Dateien `*.txt` und `*.test`; davon 174 als `.txt` und 25 als `.test`). Soll-Dateien der Dateiprüfungen (`*-ausgabe.txt`, `*_erwartet.txt`) sind keine Tests; sie liegen in `Test-Ergebnisvergleiche/` (5.4). Jeder dieser Tests hat einen `Prüfungen:`-Abschnitt, ausgenommen `bdpt1.test`, `namen_raten.test`, `pj2-bis_minimalprogramm.test`, `pj2-bis_minimalprogramm_adressdruck.test`, `pj2-bis_minimalprogramm_adresseneingabe.test`, `pj2-bis_minimalprogramm_v2.test`, `pj2-bis_minimalprogramm_v3.test`, `pj2-bis_minimalprogramm_wort_mit_kz.test`, `sw_eingabe.test`, `sw_eingabe_k2.test`, `sw_eingabe_lage.test` (Rückkehrcode 4: keine Prüfung definiert). Ein neuer Lauf wird mit den zuletzt bestätigten Sollwerten verglichen: Endzustand von Register und Flags, erwartete Speicherinhalte, erwartete Ausgaben.

Familie	Anzahl	Inhalt
NNNN-… (0001 bis 0024)	22	Befehlstests; der Dateiname nennt die Operationscodes (Laden, Addieren, Transport, Sprünge, Überlauf, Speichern)
asm_NNNN_…	7	Testaufträge aus Assemblerläufen (3.3), u. a. Vektor umspeichern, bedingte Sprünge, Sortierung
bsp_…	48	Beispielprogramme (Abschnittsnummer im Namen, z. B. bsp_6_1_4_division), darunter Tests mit Leser, Stanzer und Schreibwerk; einzelne als .test-Datei direkt aus dem Assembler
bs-…	2	Schalter BS und Flag BS getrennt: bedingter Stopp nur bei eingerastetem Schalter (bs-001), Schalter ohne Stopp ohne Wirkung (bs-002)
bdpt1.test	1	Bedienpult mit Peripherie (Test bdpt1)
k1_…	1	Leser und Stanzer an K1 (k1_leser_stanzer)
r300_…	1	R300-Eingabe: alle Zeichen mit Computertaste über SW einlesen und als Echo ausgeben (r300_eingabe, RE-001); dazu Klickspur und Sollfolge
sw_…	4	Eingabe über das Schreibwerk SW (sw_eingabe, sw_eingabe_k2, sw_eingabe_lage, sw_zwei_ziffern)
betrag.test	1	Betragsbildung (z = -|a+b| + |-b+c| + |-c+d|, erwartet z = 1)
namen_raten.test	1	Programmbeispiel Namen raten (Assembler-Projekt 1, `Assembler/Projekt_1/namen_raten`)
lb-…	3	Lochband: Irrung Zeichen übersprungen (lb-001), ohne Irrung (lb-002), Paritätsfehler (lb-003)
pj2-…	6	Programmbeispiele: ggT nach Euklid, Maximum von drei Zahlen (vier Fälle), Vertauschung zweier Zahlenfelder; dazu das Minimalprogramm des BIS als .test-Datei
pj2-bis_…	56	Tests der BIS-Bausteine (11.4): Befehlsdruck, Adressendruck, Adresseneingabe, Druck von BZ und AC; setzen das Minimalprogramm auf der Trommel voraus
s1-R1 bis s4-spur	4	Vorzeichenbit und X-Abbruch beim verkürzten Gruppen-Transport (s1-R1: Rechtsverschiebung, s2-R2: Linksverschiebung, s3-neg: AC schon vor der ersten Zelle negativ); s4-spur: Ausgabe einer ganzen Spur mit `#SPUR` (3.2)
sp-…	9	Sprungbefehle (unbedingt, zyklisch, bedingt, mit Rückkehrabsicht; Schalter SP = 0 und SP = 1)

Familie	Anzahl	Inhalt
tk-…	8	Tastenfolgen (Tabelle unten): jeder Test belegt eine Festlegung aus 4.3 bis 4.5
ac-…, bz-…, acbz-…	5	Anfangswerte `#AC=` und `#BZ=` (3.6): AC vorbelegt, BZ vorbelegt mit `G`, mit `M` und `E`, beide zusammen, mit `Ü` und `E`
int-…	6	Unterbrechung mit `#INTERRUPT` (4.7): Fortsetzung mit `H R`, Ende mit `G Ü E`
st-…	6	Makro `#STARTE` (4.6): `#STARTE 4`, Gegenprobe mit ausgeschriebener Tastenfolge, `#STARTE 12` (oktal) und `#STARTE 1000` zusammen mit der Druckroutine bzw. dem Minimalprogramm
trommel-…	1	Makro `#TROMMEL=` (2g): Trommel vor dem Test geleert (`LEER`); der Test belegt zwei Zellen selbst und prüft, dass drei weitere 0 sind (auch auf verschmutzter Trommel `Test wahr`)
wr-…	2	Lochbandeingabe von WR (Wagenrücklauf) über Leser 1 (wr-001)
ziffern_…	5	Ziffer 0 bis 3 über K0 aus Datei (`--k0`) einlesen und auswerten, Ausgabe auf dem Schreibwerk; vier Eingaben (0 bis 3) und eine Folge nicht auswertbarer Ziffern (`ziffern_uebergehen`)
Summe	199	

> **Hinweis** Die Beispieltests dieses Handbuchs (`Handbuch-Tests/`) sind nicht mitgezählt. Das Glob-Muster der Soll-Dateien muss `*-sw-ausgabe.txt` heißen, nicht `*-sw-ausgabe.*`: `bsp_seite_42_025-sw-ausgabe.test` ist ein Test und keine Soll-Datei.

Weil die Trommel ihren Inhalt behält (3.5), setzt der Gesamtlauf sie vor jedem Test zurück. Tests, die ein Programm auf der Trommel voraussetzen (die BIS-Tests und `st-004`: das Minimalprogramm, `st-005`: die Druckroutine aus `asm_druck_tastenfolge`; die Voraussetzung steht in der `#DOK`-Zeile), erhalten es durch einen vorgeschalteten Lauf (11.4). Der Lauf liefert dann für alle 199 Tests den Rückkehrcode 0 (außer `bdpt1.test` (4), `namen_raten.test` (4), `pj2-bis_minimalprogramm.test` (4), `pj2-bis_minimalprogramm_adressdruck.test` (4), `pj2-bis_minimalprogramm_adresseneingabe.test` (4), `pj2-bis_minimalprogramm_v2.test` (4), `pj2-bis_minimalprogramm_v3.test` (4), `pj2-bis_minimalprogramm_wort_mit_kz.test` (4), `sw_eingabe.test` (4), `sw_eingabe_k2.test` (4), `sw_eingabe_lage.test` (4)):

```
for t in Tests/*.txt Tests/*.test; do
  case $t in *-ausgabe.txt|*_erwartet.txt) continue;; esac
  python3 erzeuge_leere_trommel.py                # leere Trommel für jeden Test
  # Tests mit Voraussetzung: vorher das Minimalprogramm laden (BIS-Tests, st-004) bzw. die Druckroutine (st-005)
  python3 c8205_interp.py --MODUS=TEST --NO-TERMOUT --stanzer1 Test-virtuelleGeraete/Stanzer1/ "$t" > /dev/null
  echo "$? $(basename $t)"
done | awk '{print $1}' | sort | uniq -c
    188 0
     11 4     # 199 Tests, alle ohne Abweichung
```

Ergebnis dieses Laufs: alle 199 Tests ohne Abweichung (Rückkehrcode 0 = `Test wahr`, 11 × 4 ohne Prüfung).

Ohne das Zurücksetzen (die Trommel ist nur zu Beginn leer, die Tests laufen in der Reihenfolge der Schleife) ergibt derselbe Lauf 186 × Code 0, 11 × Code 4, 2 × Code 6; Test falsch: `pj2-bis_ps_zv_fehler.txt`, `st-004_STARTE_1000_druck_interrupt.txt`. Das ist die Reihenfolgeabhängigkeit aus 3.5, kein Fehler des Tests oder des Interpreters.

> **Hinweis** Der Sammellauf `compute_all_tests.sh` durchläuft nur `Tests/*.txt` (124 von 199 Tests; nicht die 25 `.test`-Dateien und nicht die 50 BIS-Tests `pj2-bis_*.txt`, die mit `test_bis.sh` laufen, 11.4). Er meldet jeden Rückkehrcode außer 0 und 4; Code 6 bleibt bei einer Testdatei mit `Prüfungen: False` ungemeldet (5.5). Die leere Trommel erzeugt er erst nach dem Lauf, nicht vor jedem Test. Für einen Gesamtlauf über alle Tests gilt die Schleife oben.

11.2 Referenztests zu den Tastenfolgen (tk-001 bis tk-008)

Die acht Tests `tk-…` stützen sich auf Festlegungen aus 4.3 bis 4.5. Die Zelle 0000 (bei tk-002 bis tk-005 auch 0001 und 0002) enthält einen Stopp-Befehl, sodass die Wirkung der Tasten im Endzustand sichtbar wird. Endzustände aus den echten Läufen (alle mit Rückkehrcode 0, `Test wahr`):

Test	Tastenfolge	BZ	AC	AR
tk-001	`H G M R`	0001	00000000000	0005
tk-002	`H G M+ E M- M R`	0002	00000004000	0010
tk-003	`H G Ü R`	0001	00000000000	0005
tk-004	`H G M+ E M- Ü R`	0002	00000002000	0010
tk-005	`H G M+ E M- ST R`	0003	00000002000	0020
tk-006	`ST R`	0001	00000000000	0005
tk-007	`Ü R`	0001	00000000000	0005
tk-008	`H G Ü E`	0001	00000000000	0005

> **Hinweis** Die Endzustände sind Sollwerte im Sinne von Kapitel 5; in jedem Test stehen sie im Abschnitt `Prüfungen:`. Die Knotenzahl der Endzeile ist dagegen fassungsabhängig und kein Sollwert.

11.3 Vollständigkeitstest

- **Vollständigkeitstest:** `c8205_befehlssweep.py` baut für alle 512 Operationscodes ein Minimalprogramm (der Befehl an Adresse 0000, der Stopp 020 an Adresse 0001) und führt es über dieselbe Knotengraph-Engine aus. Er prüft nicht die numerische Korrektheit jeder Formel, sondern nur, dass für jeden Code eine Implementierung vorhanden ist und der Automat sauber terminiert (kein Stub, kein Sackgassen-Knoten, keine erkennbare Endlosschleife).
- **Formelkorrektheit:** dafür dienen die regulären Tests mit Prüfungen (Kapitel 5), z. B. über `c8205_interpreter.sh/.bat`.
- Bei einem neuen oder geänderten Befehl zuerst den Claude-Interpreter benutzen (Kapitel 10), dann den Befehl in `c8205_interp.py` übernehmen, danach alle Referenztests und den Befehlssweep erneut laufen lassen.

```
python3 testscripte/c8205_befehlssweep.py       # alle 512 Codes, Kurzbericht
python3 testscripte/c8205_befehlssweep.py --verbose # jeden Code einzeln
python3 testscripte/c8205_befehlssweep.py --code 251 # nur ein Code (oktal), im Detail
```

11.4 Tests der BIS-Bausteine: das Minimalprogramm läuft zuerst

Ein Test, der einen Baustein des BIS prüft (z. B. ein Unterprogramm des Minimalprogramms wie Befehlsdruck, Adressendruck oder Adresseneingabe), braucht das **Minimalprogramm auf der Trommel**. Das ist für alle BIS-Tests der Normalfall und gilt als feste Regel:

1. **Schritt 1: Minimalprogramm laden.** `./c8205_interpreter.sh Tests/pj2-bis_minimalprogramm.test` belegt die Zellen 0000 bis 0337 sowie die Tabelle der Adresseneingabe auf 0400 bis 0434 (Sprung auf das Hauptprogramm in 0000, Druck von BZ und AC in 0001, die Unterprogramme ab 0040). Der Test ist beliebig oft wiederholbar; er setzt vorher nichts zurück. Wer eine saubere Ausgangslage braucht, erzeugt zuerst eine leere Trommel (`python3 erzeuge_leere_trommel.py`, 3.5).
2. **Schritt 2: der eigentliche Test.** Er liegt mit seinem Hauptprogramm üblicherweise ab Zelle 1000, ruft das Unterprogramm über seine feste Ansprungzelle auf und endet mit einem Stopp. Er belegt **keine** Zelle des Minimalprogramms (0000 bis 0337), auch nicht 0001: Eine Testzeile `0001: …` überlagert die persistente Trommel und bleibt für spätere Tests (3.5).
3. **Dokumentation im Test:** Der Test nennt die Voraussetzung in einer `#DOK`-Zeile.
4. **Wrapper:** `c8205_bis_test.sh TESTDATEI` führt Schritt 1 und Schritt 2 hintereinander aus; `test_bis.sh` setzt dazu die Trommel zurück und ruft den Wrapper für alle BIS-Tests auf.

Aufruf eines Unterprogramms aus dem Hauptprogramm: Der Wert steht in R1 (Register 7773; `211 wert` und `240 r1`), der Aufrufer wählt den Ausgabekanal an (`030 4500` für K2; die Ausgabeunterprogramme enthalten keinen Anwahlbefehl), dann folgt `025 ansprungzelle`. Die Rückkehr erfolgt in die Zelle hinter dem Aufruf.

Echtes Beispiel `Tests/pj2-bis_befehlsdruck.txt`: Das Hauptprogramm holt den Befehl aus Zelle 1000 (`211 1000`, Wort `00001000211`) in den AC, speichert ihn nach R1 und ruft den Befehlsdruck (`025 053`). Erwartet wird die Schreibwerk-Ausgabe `1000 211` (ohne Zeilenende), R1 bleibt unverändert, Stopp in 1004 mit BZ=1005. V1=V2=1 stammen aus dem letzten Vergleich der Ziffernschleife, nicht aus dem Hauptprogramm. Die `#DOK`-Texte sind gekürzt:

```
Trommeldatei:
#DOK-0001 …
#PROT STUFE=2
#STARTE 1000
--sw-ausgabe Test-virtuelleGeraete/SW-Ausgabe

# Hauptprogramm (die Zellen des Minimalprogramms 0000 bis 0337 werden nicht belegt)
1000: 00001000211  # 211 1000 ; AC := Zelle 1000 (der Befehl selbst: 1000 211)
1001: 00007773240  # 240 r1 ; R1 := Befehl
1002: 00004500030  # 030 4500 ; Kanal K2 (Schreibwerk) anwaehlen
1003: 00000053025  # 025 053 ; Befehlsdruck
1004: 00000000020  # 020 0 ; Stopp

Prüfungen:
BZ=1005 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=1 V2=1
7773: 00001000211  # R1 bleibt unveraendert
0001: 00000150000  # Zelle 0001 bleibt (Sprung nach druck)
--pruefe-sw-ausgabe Test-Ergebnisvergleiche/pj2-bis_befehlsdruck_erwartet.txt
```

```
$ python3 erzeuge_leere_trommel.py
$ python3 c8205_interp.py --MODUS=TEST Tests/pj2-bis_minimalprogramm.test          # Schritt 1 (Rückkehrcode 4:
    keine Prüfung)
$ python3 c8205_interp.py --MODUS=TEST Tests/pj2-bis_befehlsdruck.txt               # Schritt 2
=== Test pj2-bis_befehlsdruck (#PROT STUFE=2) ===
…
Ende: Unbedingter Stop (US:=1) | Knoten: 2630 | BZ=1005 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=1
    V2=1
…
Ergebnispruefung:
… (übrige Zeilen alle OK)
  Schreibwerk: soll=Test-Ergebnisvergleiche/pj2-bis_befehlsdruck_erwartet.txt
    ist=Test-virtuelleGeraete/SW-Ausgabe/sw-ausgabe.txt OK
Test wahr

$ cat Test-Ergebnisvergleiche/pj2-bis_befehlsdruck_erwartet.txt
1000 211
Rückkehrcode Schritt 2: 0
```

Die zehn Tests `pj2-bis_*`**.** Je Test eine Zeile mit dem Ergebnis eines echten Laufs auf leerer Trommel (Schritt 1, danach der Test); alle zehn sind `Test wahr`:

Test	BZ	AC	Rückkehrcode
adressendruck	1005	01234567012	0
adresseneingabe_4stellig	1006	00001234000	0
adresseneingabe_8stellig	1006	12345670000	0
adresseneingabe_bm_anfang	0042	00000000005	0
adresseneingabe_buchstabe_wie_null	1003	00000010000	0
adresseneingabe_gb_in_ziffern	0042	00000000005	0
adresseneingabe_null	1006	00000000000	0
adresseneingabe_ueberzaehlig	1006	12345670000	0
adresseneingabe_wr_anfang	0042	00000000005	0
adresseneingabe_ziffer8_halt	0260	10000012000	0
adresseneingabe_ziffer9_erste	0260	10000000000	0
adresseneingabe_zr	1011	00000000005	0
adresseneingabe_zv_wie_null	1003	00000010000	0
befehlsdruck	1005	00000000000	0
druck_bz_ac	1006	01234567012	0
kz_bm_vor_k1	0042	00000000005	0

Test	BZ	AC	Rückkehrcode
kz_doppelpunkt_im_wort	0042	00000000005	0
kz_gb_im_wort	0042	00000000005	0
kz_gb_vor_k1	0042	00000000005	0
kz_irrung_zeichen_im_wort	1003	00000000012	0
kz_irrung_zeichen_vor_k1	1003	00000000005	0
kz_is_im_wort	0042	00000000005	0
kz_is_vor_k1	0042	00000000005	0
kz_k1_1_punkt	0042	00000000005	0
kz_k1_2_punkt	0042	00000000005	0
kz_kb_im_wort	0042	00000000005	0
kz_kb_vor_k1	0042	00000000005	0
kz_null_punkt	1003	00000000000	0
kz_wort_basis	1003	00000000005	0
kz_wr_im_wort	0042	00000000005	0
kz_wr_vor_k1	0042	00000000005	0
kz_wr_zwischen_k1_k2	0042	00000000005	0
kz_zr_im_wort	0042	00000000005	0
kz_zr_vor_k1	0042	00000000005	0
kz_zv_vor_k1	0042	00000000005	0
ps_adressaenderung_k2	1003	00000000000	0
ps_hla_auf_band	1003	00000000000	0
ps_hla_neu_mitten	1003	00000000000	0
ps_hla_vorbesetzt	1003	00000000000	0
ps_null_punkt	1003	00000000000	0
ps_relativadresse	1003	00000000000	0
ps_schlusszeichen_bm	1003	00000000000	0
ps_schlusszeichen_sofort	1003	00000000000	0
ps_sofortbefehl_sprung	1051	00001050000	0
ps_sofortbefehl_start	0552	00000000006	0
ps_sofortbefehl_us	0552	00000000005	0
ps_unzulaessiges_zeichen	0042	00000000005	0
ps_wr_im_wort	1003	00000000000	0
ps_zeichen_zwischen_woertern	1003	00000000000	0
ps_zv_fehler	0042	00000000007	0

Eingaben der Adresseneingabe. Je Test eine Lochstreifendatei `Test-virtuelleGeraete/Leser1/pj2-bis_adresseneingabe_<Name>.txt` (Ansprungzelle 0316) mit den Zeichen im R-300-Code, eine Zeile je Zeichen im Format `w v u p 8 4 2 1` (9.1). Das Hauptprogramm ab 1000 wählt K1 an, ruft die Adresseneingabe, speichert den AC nach R1 und druckt ihn mit dem Adressendruck (`030 4500`, `025 0100`); es werden nur die mittleren vier Oktalziffern gedruckt (Wortstellen z12 bis z23).

Test	Eingabe	Erwartet
`_4stellig`	`1234.`	R1 = `00001234000`, Schreibwerk `1234`, Stopp 1005, BZ=1006
`_8stellig`	`12345670.`	R1 = `12345670000`, Schreibwerk `5670` (die mittleren vier Ziffern), BZ=1006

Test	Eingabe	Erwartet
`_null`	`.`	R1 = 0 (die Konstante Null braucht nur das Schlusszeichen), Schreibwerk `0000`, BZ=1006
`_zr`	`1 ZR 0 0 5 .`	ZR bricht die Adresseneingabe ab und springt nach `kz_neu` (Zelle 0412); die neu gestartete Kennzeicheneingabe liest K_1 = 0, K_2 = 0 und das Wort 5 und kehrt über `kz_rs` (7700) nach 1010 zurück; Stopp, BZ=1011, AC = `00000000005`
`_ueberzaehlig`	`1234567091.`	ab der neunten Ziffer wird alles ignoriert (auch die 9): R1 = `12345670000`, Schreibwerk `5670`, BZ=1006
`_ziffer8_halt`	`128.`	die 8 ist keine Oktalziffer: HALT im Befehl `ae_halt` (Zelle 0257), BZ=0260, AC = `10000012000`
`_ziffer9_erste`	`9`	auch als erstes Zeichen HALT: BZ=0260, AC = `10000000000`, V1=V2=1

Hinweis Verlegung der Arbeitszellen (11.10.2026): Die Arbeitszellen der Ausgabe- und Eingabeunterprogramme lagen auf 7554 bis 7563, den Speicherplätzen der Ansprungzellen der anderen Bausteine. Sie liegen jetzt auf 7740 bis 7744 und 7756 bis 7760 (Spezifikation `C8205-Interpreter-BIS.md`, BIS-A.3); die Zellen 7554 bis 7563 bleiben nach allen Läufen null. Die Läufe oben sind mit dem neu assemblierten Minimalprogramm entstanden.

12 Checkliste und Vorlage

12.1 Checkliste für neue Tests

1. Programm endet mit einem Stopp-Befehl (`00000000020`)?
2. Adressen vierstellig, Werte elfstellig, nur Ziffern 0–7?
3. Keine Adresse doppelt mit verschiedenem Wert?
4. Alle Zellen, auf die sich das Ergebnis stützt (gelesen oder geprüft), im Trommelabschnitt genannt, auch die mit Wert 0? Sonst ist der Test nur auf leerer Trommel reproduzierbar (3.5).
5. Messpunkt gesetzt (`164 xxxx`) und passende `#DOK`/`#SPEI`/`#SPUR` vorhanden?
6. `#PROT STUFE=` bewusst gewählt (Schleifen: 0 oder 2)?
7. Tastenfolge endet mit R oder im Halt; `SW=1` vor G, wenn K0 mit Schreibwerk gebraucht wird?
8. Wenn das Programm Zeichen liest oder schreibt: Kanal angewählt und das Gerät zugeordnet (Schalter oder Zeile `--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe`, `--k0` in der Testdatei)?
9. Prüfungen: mit den wichtigsten Sollwerten, damit der Test als Regressionstest taugt; bei Stanzer oder Schreibwerk zusätzlich die Dateiprüfung (`--pruefe-stanzer1`, `--pruefe-sw-ausgabe`)?
10. Anfangswerte der Register nötig? Dann `#AC=` (11 Ziffern) und `#BZ=` (4 Ziffern); beachten, dass `G` den BZ nicht setzt (3.6). Ansprung eines Speicherplatzes: `#STARTE n` statt einer langen Tastenfolge (4.6).
11. `#INTERRUPT` nur zwischen zwei Befehlszeilen, mit `H` als erster Taste, und höchstens einmal je Befehlszeile (4.7)?
12. Soll die Prüfung bewusst `Test falsch` ergeben: Kopfzeile `Prüfungen: False` (5.5)? Soll-Dateien der Dateiprüfungen in `Test-Ergebnisvergleiche/` ablegen, mit dem Namensende `-ausgabe.txt` oder `_erwartet.txt` (5.4).
13. Test eines BIS-Bausteins: Minimalprogramm vorher laden (`c8205_bis_test.sh`), keine Zelle 0000 bis 0337 belegen und die Voraussetzung in einer `#DOK`-Zeile nennen (11.4)?

12.2 Vorlage

Die Vorlage ist mit dem Python-Interpreter lauffähig (`Handbuch-Tests/vorlage.txt`) und entspricht dem Test aus Kapitel 5; bei Stufe 2 entsteht das Protokoll nach `Test-Protokolle/<Testname>/protokoll.txt`.

```
Trommeldatei:
#DOK-0001 Kurzbeschreibung des Tests
#SPEI-0001 3000-3001
#PROT STUFE=2
# Programm
0000: 00003000211 # AC := <3000>
0001: 00003001210 # AC := AC + <3001>
0002: 00000001164 # Leerbefehl 164 -> Diagnose #DOK-0001
0003: 00000000020 # Stopp
# Daten
3000: 00000000100
3001: 00000000045

Tastenfolge:
H G R

Prüfungen:
BZ=0004 AC=00000000145 US=1
3001: 00000000045
```

Der echte Lauf der Vorlage (Rückkehrcode 0):

```
$ python3 c8205_interp.py --MODUS=TEST Handbuch-Tests/vorlage.txt
=== Test vorlage (#PROT STUFE=2) ===
-- Diagnose --
#DOK-0001 Kurzbeschreibung des Tests
#SPEI-0001 3000-3001
3000: 00000000100
3001: 00000000045
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0
    V2=0
Ergebnispruefung:
  BZ: soll=0004 ist=0004 OK
  AC: soll=00000000145 ist=00000000145 OK
  US: soll=1 ist=1 OK
  3001: soll=00000000045 ist=00000000045 OK
Test wahr
Protokolldatei: Test-Protokolle/vorlage/protokoll.txt
```

Anhang A: Begriffe

Begriff	Erklärung
Test	ein simulierter Rechnerlauf: Trommeldatei + Tastenfolge (+ Prüfungen)
Trommeldatei	zwei Bedeutungen: der Abschnitt `Trommeldatei:` einer Testdatei (Trommelabschnitt: `Adresse: Wert`-Zeilen, Testmarken und Geräte-Zeilen) und die Datei `trommel.txt` des Python-Interpreters mit dem ganzen Speicher (3.5)
Persistente Trommel	die Datei `trommel.txt` (4096 Zeilen); behält ihren Inhalt über Tests und Aufrufe, wird nur durch `erzeuge_leere_trommel.py` zurückgesetzt
Überlagerung	Der Interpreter schreibt vor dem Start die Zeilen des Trommelabschnitts auf die Trommel; nicht genannte Zellen behalten ihren Wert
Tastenfolge	Bedienhandlungen am Pult, die den Ausgangszustand von Registern und Flags festlegen
Testmarke	Zeile mit `#`, die nur der Testdurchführung dient (`#DOK`, `#SPEI`, `#SPUR`, `#PROT`, `#AC=`, `#BZ=`, `#INTERRUPT`, `#STARTE`)
`#STARTE n`	Makro für den Ansprung des Speicherplatzes n; ersetzt die Tastenfolge `H G M+ E … E M- R` (4.6)
`#INTERRUPT`	Zeile zwischen zwei Befehlszeilen: unterbricht den Lauf nach dem Befehl davor und bedient eine weitere Tastenfolge (4.7)
`Prüfungen: False`	Kopfzeile eines Tests, der bewusst `Test falsch` ergeben soll; der Rückkehrcode bleibt 6, die Sammelläufe melden ihn nicht (5.5)
Minimalprogramm (BIS)	Grundprogramm auf der Trommel (Zellen 0000 bis 0337 und 0400 bis 0434); jeder Test eines BIS-Bausteins setzt es voraus (11.4)
Geräte-Zeile	Zeile der Testdatei mit `--` (`--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe`, `--k0`), die ein virtuelles Gerät zuordnet

Begriff	Erklärung
Dateiprüfung	Prüfung, die die Stanzer- oder Schreibwerk-Ausgabedatei mit einer Soll-Datei vergleicht (`--pruefe-stanzer1`, `--pruefe-sw-ausgabe`)
Leerbefehl 164	Operation B = 1, A = 6, Z = 4: tut nichts (Messpunkt); löst bei passender Adresse die Diagnose aus
Konnektor B	Eintritt in den Ablauf von außen (Knoten 1 des Knotengraphen)
Knoten 19	Ende der Phase-III-Arbeit; Startpunkt bei aufgestelltem Befehlsregister (nach Ü, ST, M ohne E oder E ohne M)
Knoten 54 (Konnektor 4)	Startpunkt nach M mit Sprung mit Rückkehrabsicht (025, 027, 125, 127): Phase III gilt als abgelaufen
Knotenautomat	der Knotengraph des Befehlsablaufs mit seinen vier Knotentypen (Konnektor, Phasenzuordnung, Anweisung, Test)
Stub	Meldung „an dieser Stelle fehlt eine Umsetzung“; der Test wird angehalten
Regressionstest	ein Test, dessen Sollwerte bestätigt sind und der bei jeder Änderung erneut gegen sie verglichen wird
Rückkehrcode	Ergebnis eines Interpreterlaufs für das Betriebssystem: 0 wahr, 4 ohne Prüfung, 6 falsch, 8 Laufzeitfehler, 16 Vorab-Prüfung negativ oder Trommeldatei fehlerhaft, 2 ungültiger Aufruf
Claude-Interpreter	Verfahren, bei dem eine KI den Knotengraphen von Hand abarbeitet (Gegenprobe, neue Befehle)
Interleaving	logische Reihenfolge der Sektoren einer Spur ≠ physische; Schritt +05 (oktal) von logisch L nach L+1

Anhang B: Verweise

Thema	Dokument
Aufruf, Schalter, Fehlermeldungen des Interpreters	PDF „Bedienungsanleitung Python-Interpreter“
Rechner, Phasen, Register, Trommel	PDF „Rechnerbeschreibung“
Tasten, Schalter, Geräte	PDF „Bedienpult und Peripherie“
Alle Operationscodes	PDF „Einzelbefehlsliste“
Zeichenkodes (Tabelle 1)	PDF „Kodetabellen“
Knotengraph und T2	PDF „Knotengraph“
Assembler, .test-Datei	PDF „Assembler-Dokumentation“